

ux_design_spec

HelixCode CLI — Enterprise-Grade Model Display UX Design Specification

Version: 1.0.0

Date: 2026-04-30

Scope: LLM Model Display UX for HelixCode CLI across all 7 supported platforms

Target Files: cmd/cli/main.go, internal/cli/ux/, internal/cli/model_display.go (new)

Dependencies: github.com/fatih/color v1.18.0, github.com/rivo/tview v0.42.0

Data Source: LLMsVerifier (via REST API at http://localhost:8081/api/v1/verifier)

Table of Contents

1. [Cross-Platform Symbol Strategy](#)
 2. [Color System](#)
 3. [Status Indicators & Badges](#)
 4. [Model List Display \(--list-models\)](#)
 5. [Model Detail Display \(--model-info <id>\)](#)
 6. [Interactive Model Selection](#)
 7. [Notification / Alert UX](#)
 8. [Real-time Updates Display](#)
 9. [Error / Empty States](#)
 10. [Go Structs for UX State Management](#)
 11. [CLI Flag Additions](#)
 12. [Files to Modify / Create](#)
 13. [Implementation Priority](#)
-

1. Cross-Platform Symbol Strategy

1.1 Platform Detection

```
// internal/cli/ux/symbols.go
type TerminalCapabilities struct {
    SupportsEmoji    bool    // true for macOS Terminal, iTerm2,
                modern Linux, WSL
    SupportsUnicode  bool    // true for all except Windows
                cmd.exe pre-Windows 11
    Supports256Color bool    // true for virtually all modern
                terminals
    SupportsTrueColor bool    // true for iTerm2, Windows
                Terminal, modern Linux
```

```

Width                int    // terminal width in columns
IsWindowsCMD         bool    // Windows cmd.exe (not PowerShell,
                             not WSL)
IsPowerShell         bool    // Windows PowerShell / pwsh
IsMobile             bool    // Android/iOS terminal (limited
                             width)
}

func DetectTerminalCapabilities() *TerminalCapabilities {
    // Detection logic:
    // 1. Check runtime.GOOS
    // 2. Check TERM env var
    // 3. Check WT_SESSION (Windows Terminal)
    // 4. Check iTerm (ITERM_SESSION_ID)
    // 5. Check COLORTERM=truecolor
    // 6. Use github.com/mattn/go-isatty for TTY detection (add
        to deps if needed)
    // 7. Use tput or stty for width on Unix;
        GetConsoleScreenBufferInfo on Windows
}

```

1.2 Symbol Sets by Platform

Category	Symbol (Rich)	Fallback (ASCII)	Windows CMD	PowerShell	Mobile
Verified	✓ U+2713	[OK]	[OK]	✓	[V]
Pending	⌚ U+23F3	[. .]	[. .]	⌚	[P]
Failed	✗ U+2717	[XX]	[XX]	✗	[X]
Not Tested	⊘ U+2298	[-]	[-]	⊘	[-]
Healthy	● U+25CF	[+]	[+]	●	[+]
Degraded	◐ U+25D0	[~]	[~]	◐	[~]
Unhealthy	● U+25CF red	[-]	[-]	●	[-]
Offline	○ U+25CB	[!]	[!]	○	[!]
Rate Limited	⏸ U+23F8	[RL]	[RL]	⏸	[RL]
Quota Exceeded	🚫 U+26D4	[QU]	[QU]	🚫	[QU]
Cool Down	🕒 U+1F552	[CD]	[CD]	🕒	[CD]
Star / Score	★ U+2605	*	*	★	*
Empty Star	☆ U+2606	-	-	☆	-
Vision	👁 U+1F441	[VSN]	[V]	👁	[V]
Audio	🔊 U+1F50A	[AUD]	[A]	🔊	[A]

Category	Symbol (Rich)	Fallback (ASCII)	Windows CMD	PowerShell	Mobile
Video	 U+1F3AC	[VID]	[V]		[V]
Reasoning	 U+1F9E0	[RSN]	[R]		[R]
Streaming	 U+26A1	[STR]	[S]		[S]
Tools	 U+1F527	[TOL]	[T]		[T]
Code	</> U+003C/	[COD]	[C]	</>	[C]
Embeddings	 U+1F4CA	[EMB]	[E]		[E]
Open Source	 U+1F513	[OSS]	[O]		[O]
Arrow Right	→ U+2192	- >	- >	→	>
Arrow Up	↑ U+2191	^	^	↑	^
Bullet	• U+2022	-	-	•	-
Diamond	◆ U+25C6	>	>	◆	>
Separator	U+2502				
Horizontal	— U+2500	-	-	—	-
Corner	┐ U+250C	+	+	┐	+
Progress	■ U+2588	#	#	■	#
Progress Empty	░ U+2591	.	.	░	.
Dollar	\$ U+0024	\$	\$	\$	\$
Latency Fast	 U+1F680	[FAST]	[F]		[F]
Latency Normal	 U+1F6B6	[NRM]	[N]		[N]
Latency Slow	 U+1F40C	[SLO]	[S]		[S]

1.3 Symbol Set Implementation

```
// internal/cli/ux/symbols.go
```

```
type SymbolSet struct {
    Verified      string
    Pending       string
    Failed        string
    NotTested     string
    Healthy       string
    Degraded      string
    Unhealthy     string
}
```

```

Offline          string
RateLimited      string
QuotaExceeded    string
CoolDown         string
StarFilled       string
StarEmpty        string
Vision           string
Audio            string
Video            string
Reasoning        string
Streaming        string
Tools            string
Code             string
Embeddings       string
OpenSource       string
ArrowRight       string
ArrowUp          string
Bullet           string
Diamond          string
SepVertical      string
SepHorizontal    string
CornerTL         string
ProgressFull     string
ProgressEmpty    string
Dollar           string
LatencyFast      string
LatencyNormal    string
LatencySlow      string
}

func NewSymbolSet(cap *TerminalCapabilities) *SymbolSet {
    if cap.IsWindowsCMD && !cap.SupportsEmoji {
        return &SymbolSet{
            Verified: "[OK]", Pending: "[..]", Failed: "[XX]",
            NotTested: "[-]", Healthy: "[+]", Degraded: "[~]",
            Unhealthy: "[-]", Offline: "[!]", RateLimited:
                "[RL]",
            QuotaExceeded: "[QU]", CoolDown: "[CD]",
            StarFilled: "*", StarEmpty: "-",
            Vision: "[V]", Audio: "[A]", Video: "[V]",
            Reasoning: "[R]", Streaming: "[S]", Tools: "[T]",
            Code: "[C]", Embeddings: "[E]", OpenSource: "[O]",
            ArrowRight: "->", ArrowUp: "^", Bullet: "-",
            Diamond: ">", SepVertical: "|", SepHorizontal: "-",
            CornerTL: "+", ProgressFull: "#", ProgressEmpty: ".",
            Dollar: "$", LatencyFast: "[F]", LatencyNormal:
                "[N]",
            LatencySlow: "[S]",
        }
    }
}

```

```

    }
}
if cap.IsMobile {
    return &SymbolSet{
        Verified: "[V]", Pending: "[P]", Failed: "[X]",
        NotTested: "[-]", Healthy: "[+]", Degraded: "[~]",
        Unhealthy: "[-]", Offline: "[!]", RateLimited:
        "[RL]",
        QuotaExceeded: "[QU]", CoolDown: "[CD]",
        StarFilled: "*", StarEmpty: "-",
        Vision: "[V]", Audio: "[A]", Video: "[V]",
        Reasoning: "[R]", Streaming: "[S]", Tools: "[T]",
        Code: "[C]", Embeddings: "[E]", OpenSource: "[O]",
        ArrowRight: ">", ArrowUp: "^", Bullet: "-",
        Diamond: ">", SepVertical: "|", SepHorizontal: "-",
        CornerTL: "+", ProgressFull: "#", ProgressEmpty: ".",
        Dollar: "$", LatencyFast: "[F]", LatencyNormal:
        "[N]",
        LatencySlow: "[S]",
    }
}
// Rich Unicode set (macOS, Linux, iTerm2, Windows Terminal,
// WSL, PowerShell)
return &SymbolSet{
    Verified: "✓", Pending: "⌚", Failed: "✗",
    NotTested: "⊘", Healthy: "●", Degraded: "◐",
    Unhealthy: "●", Offline: "○", RateLimited: "⏹",
    QuotaExceeded: "🚫", CoolDown: "🕒",
    StarFilled: "★", StarEmpty: "☆",
    Vision: "🔍", Audio: "🔊", Video: "🎬",
    Reasoning: "🧠", Streaming: "⚡", Tools: "🔧",
    Code: "</>", Embeddings: "📊", OpenSource: "🔒",
    ArrowRight: "→", ArrowUp: "↑", Bullet: "•",
    Diamond: "◆", SepVertical: "|", SepHorizontal: "—",
    CornerTL: "└", ProgressFull: "■", ProgressEmpty: "░",
    Dollar: "$", LatencyFast: "🚀", LatencyNormal: "🚶",
    LatencySlow: "🐌",
}
}

```

2. Color System

2.1 Color Constants (fatih/color)

```

// internal/cli/ux/colors.go
package ux

```

```

import "github.com/fatih/color"

var (
    // Primary colors
    CHeader      = color.New(color.FgHiCyan, color.Bold)
    CSubheader   = color.New(color.FgCyan)
    CLabel       = color.New(color.FgHiBlack)
    CValue       = color.New(color.FgWhite)
    CAccent      = color.New(color.FgHiMagenta, color.Bold)

    // Status colors
    CVerified    = color.New(color.FgHiGreen, color.Bold)
    CPending     = color.New(color.FgHiYellow)
    CFailed      = color.New(color.FgHiRed, color.Bold)
    CNotTested   = color.New(color.FgHiBlack)
    CHealthy     = color.New(color.FgHiGreen)
    CDegraded    = color.New(color.FgHiYellow)
    CUnhealthy   = color.New(color.FgHiRed)
    COffline     = color.New(color.FgHiBlack)

    // Score colors
    CScoreExcellent = color.New(color.FgHiGreen,
        color.Bold) // 9.0-10.0
    CScoreGood      = color.New(color.FgGreen) // 7.0-8.9
    CScoreAverage   = color.New(color.FgYellow) // 5.0-6.9
    CScorePoor      = color.New(color.FgHiRed) // 3.0-4.9
    CScoreBad       = color.New(color.FgHiBlack) // 0.0-2.9

    // Price colors
    CPriceCheap    = color.New(color.FgHiGreen) // < $0.50/1K
    CPriceModerate = color.New(color.FgYellow) // $0.50-
    CPriceExpensive = color.New(color.FgHiRed) // >
    CPriceFree     = color.New(color.FgHiCyan, color.Bold) // $0

    // Capability colors
    CCapEnabled = color.New(color.FgHiGreen)
    CCapDisabled = color.New(color.FgHiBlack)

    // Alert colors
    CAlertInfo    = color.New(color.FgHiBlue, color.Bold)
    CAlertWarning = color.New(color.FgHiYellow, color.Bold)
    CAlertError   = color.New(color.FgHiRed, color.Bold)

```

```

CAAlertSuccess = color.New(color.FgHiGreen, color.Bold)

// UI elements
CBorder = color.New(color.FgHiBlack)
CBarExcellent = color.New(color.BgHiGreen, color.FgBlack)
CBarGood = color.New(color.BgGreen, color.FgBlack)
CBarAverage = color.New(color.BgYellow, color.FgBlack)
CBarPoor = color.New(color.BgHiRed, color.FgBlack)
CBarEmpty = color.New(color.BgHiBlack)

// Cooldown / rate limit
CCoolDown = color.New(color.FgHiYellow, color.Bold)
CRateLimit = color.New(color.FgHiRed, color.Bold)

// Provider-specific colors (for quick visual recognition)
CProviderOpenAI = color.New(color.FgGreen)
CProviderAnthropic = color.New(color.FgCyan)
CProviderGemini = color.New(color.FgBlue)
CProviderXAI = color.New(color.FgHiBlack)
CProviderGroq = color.New(color.FgMagenta)
CProviderOllama = color.New(color.FgHiYellow)
CProviderLocal = color.New(color.FgWhite)
CProviderOther = color.New(color.FgHiWhite)
)

// GetScoreColor returns the appropriate color for a 0.0-10.0
score
func GetScoreColor(score float64) *color.Color {
    switch {
    case score >= 9.0: return CScoreExcellent
    case score >= 7.0: return CScoreGood
    case score >= 5.0: return CScoreAverage
    case score >= 3.0: return CScorePoor
    default: return CScoreBad
    }
}

// GetPriceColor returns the appropriate color for a price per 1K
tokens
func GetPriceColor(price float64) *color.Color {
    if price == 0 { return CPriceFree }
    if price < 0.5 { return CPriceCheap }
    if price < 2.0 { return CPriceModerate }
    return CPriceExpensive
}

// GetProviderColor returns brand color for known providers
func GetProviderColor(provider string) *color.Color {

```

```

switch provider {
case "openai", "gpt":      return CProviderOpenAI
case "anthropic", "claude": return CProviderAnthropic
case "gemini", "google":   return CProviderGemini
case "xai", "grok":        return CProviderXAI
case "groq":               return CProviderGroq
case "ollama":             return CProviderOllama
case "local", "llamacpp", "vllm", "localai": return
    CProviderLocal
default:                   return CProviderOther
}
}

```

3. Status Indicators & Badges

3.1 Badge Rendering System

```

// internal/cli/ux/badges.go

type Badge struct {
    Symbol string
    Text   string
    Color  *color.Color
}

func (b *Badge) Render(sym *SymbolSet, width int) string {
    if width >= 100 {
        return b.Color.Sprintf("%s %s", b.Symbol, b.Text)
    }
    return b.Color.Sprintf("%s", b.Symbol)
}

func VerificationBadge(status string, sym *SymbolSet) *Badge {
    switch status {
    case "verified":
        return &Badge{Symbol: sym.Verified, Text: "VERIFIED",
            Color: CVerified}
    case "pending":
        return &Badge{Symbol: sym.Pending, Text: "PENDING",
            Color: CPending}
    case "failed":
        return &Badge{Symbol: sym.Failed, Text: "FAILED", Color:
            CFailed}
    default:
        return &Badge{Symbol: sym.NotTested, Text: "NOT TESTED",
            Color: CNotTested}
    }
}

```



```

}

func ProviderHealthBadge(status string, sym *SymbolSet) *Badge {
    switch status {
    case "healthy":
        return &Badge{Symbol: sym.Healthy, Text: "HEALTHY",
            Color: CHealthy}
    case "degraded":
        return &Badge{Symbol: sym.Degraded, Text: "DEGRADED",
            Color: CDegraded}
    case "unhealthy":
        return &Badge{Symbol: sym.Unhealthy, Text: "UNHEALTHY",
            Color: CUnhealthy}
    case "offline":
        return &Badge{Symbol: sym.Offline, Text: "OFFLINE",
            Color: COffline}
    default:
        return &Badge{Symbol: sym.NotTested, Text: "UNKNOWN",
            Color: CNotTested}
    }
}

```

```

func CooldownBadge(reason string, resetTime time.Time, sym
    *SymbolSet) *Badge {
    timeLeft := time.Until(resetTime)
    timeStr := ""
    if timeLeft > 0 {
        if timeLeft < time.Minute {
            timeStr = fmt.Sprintf(" (%ds)",
                int(timeLeft.Seconds()))
        } else if timeLeft < time.Hour {
            timeStr = fmt.Sprintf(" (%dm)",
                int(timeLeft.Minutes()))
        } else {
            timeStr = fmt.Sprintf(" (%dh)",
                int(timeLeft.Hours()))
        }
    }
    switch reason {
    case "rate-limited":
        return &Badge{Symbol: sym.RateLimited, Text: "RATE
            LIMITED" + timeStr, Color: CRateLimit}
    case "quota-exceeded":
        return &Badge{Symbol: sym.QuotaExceeded, Text: "QUOTA
            EXCEEDED" + timeStr, Color: CAlertError}
    case "cooldown":
        return &Badge{Symbol: sym.CoolDown, Text: "COOLDOWN" +
            timeStr, Color: CCoolDown}
    }
}

```

```

    default:
        return &Badge{Symbol: sym.CoolDown, Text: "UNAVAILABLE"
            + timeStr, Color: CAlertError}
    }
}

func ScoreBadge(score float64, sym *SymbolSet) string {
    c := GetScoreColor(score)
    // Visual bar: 10 chars = 10.0 score
    filled := int(score)
    if filled > 10 { filled = 10 }
    if filled < 0 { filled = 0 }
    bar := strings.Repeat(sym.ProgressFull, filled) +
        strings.Repeat(sym.ProgressEmpty, 10-filled)
    return c.Sprintf("%.1f %s", score, bar)
}

func TierBadge(tier int, sym *SymbolSet) string {
    // Tier 1=Premium, 2=High-quality, 3=Fast, 4=Aggregator,
    // 5=Free
    filled := 6 - tier // 5 stars for tier 1, 1 star for tier 5
    if filled < 1 { filled = 1 }
    if filled > 5 { filled = 5 }
    stars := strings.Repeat(sym.StarFilled, filled) +
        strings.Repeat(sym.StarEmpty, 5-filled)
    return CAccent.Sprintf("%s", stars)
}

func PriceBadge(inputPrice, outputPrice float64, sym *SymbolSet,
    width int) string {
    if inputPrice == 0 && outputPrice == 0 {
        return CPriceFree.Sprintf("%s FREE", sym.Dollar)
    }
    // Show combined per-1K price
    avgPrice := (inputPrice + outputPrice) / 2.0 * 1000 //
        convert to per-1K
    c := GetPriceColor(avgPrice)
    if width >= 100 {
        return c.Sprintf("%s%.2f/1K", sym.Dollar, avgPrice)
    }
    return c.Sprintf("%s%.1f", sym.Dollar, avgPrice)
}

```

3.2 Capability Icon Strip

```
// internal/cli/ux/capabilities.go
```

```

func CapabilityStrip(m *UnifiedModel, sym *SymbolSet, width int)
    string {
    parts := []string{}

    if m.SupportsVision {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Vision))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Vision))
    }

    if m.SupportsStreaming {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Streaming))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Streaming))
    }

    if m.SupportsTools {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Tools))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Tools))
    }

    // Code capability from verification results
    if slices.Contains(m.Capabilities, "code-generation") {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Code))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Code))
    }

    // Reasoning
    if slices.Contains(m.Capabilities, "reasoning") {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Reasoning))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Reasoning))
    }

    if width >= 100 {
        return strings.Join(parts, " ")
    }
}

```

```

}
// Narrow: only show enabled caps, no gaps
enabledOnly := []string{}
for _, p := range parts {
    if !strings.Contains(p, strings.TrimSpace(sym.Vision))
        && // crude check - better to track separately
}
// Better approach: build two slices
return strings.Join(parts, "")
}

```

// Better implementation:

```

func CapabilityStripCompact(m *UnifiedModel, sym *SymbolSet)
    string {
    caps := []struct{
        enabled bool
        symbol   string
        label    string
    }{
        {m.SupportsVision, sym.Vision, "vision"},
        {m.SupportsStreaming, sym.Streaming, "streaming"},
        {m.SupportsTools, sym.Tools, "tools"},
        {slices.Contains(m.Capabilities, "code-generation"),
        sym.Code, "code"},
        {slices.Contains(m.Capabilities, "reasoning"),
        sym.Reasoning, "reasoning"},
        {slices.Contains(m.Capabilities, "embeddings"),
        sym.Embeddings, "embeddings"},
        {m.OpenSource, sym.OpenSource, "oss"},
    }

    parts := []string{}
    for _, c := range caps {
        if c.enabled {
            parts = append(parts, CCapEnabled.Sprintf("%s",
            c.symbol))
        }
    }
    return strings.Join(parts, " ")
}

```

```

func CapabilityStripFull(m *UnifiedModel, sym *SymbolSet) string
{
    caps := []struct{
        enabled bool
        symbol   string
        label    string
    }{

```

```

    {m.SupportsVision, sym.Vision, "vision"},
    {m.SupportsAudio, sym.Audio, "audio"},
    {m.SupportsVideo, sym.Video, "video"},
    {m.SupportsStreaming, sym.Streaming, "streaming"},
    {m.SupportsTools, sym.Tools, "tools"},
    {m.SupportsFunctions, sym.Tools, "functions"},
    {slices.Contains(m.Capabilities, "code-generation"),
    sym.Code, "code"},
    {slices.Contains(m.Capabilities, "reasoning"),
    sym.Reasoning, "reasoning"},
    {slices.Contains(m.Capabilities, "embeddings"),
    sym.Embeddings, "embeddings"},
    {m.OpenSource, sym.OpenSource, "oss"},
}

parts := []string{}
for _, c := range caps {
    if c.enabled {
        parts = append(parts, CCapEnabled.Sprintf("%s %s",
        c.symbol, c.label))
    } else {
        parts = append(parts, CCapDisabled.Sprintf("%s %s",
        c.symbol, c.label))
    }
}
return strings.Join(parts, " ")
}

```

4. Model List Display (--list-models)

4.1 CLI Flags to Add

--list-models	List available models (existing)
--provider <name>	Filter by provider (new)
--verified-only	Show only verified models (new)
--max-price <float>	Max price per 1K tokens (new)
--min-score <float>	Min overall score 0-10 (new)
--capability <name>	Filter by capability: vision,streaming,too
--sort <field>	Sort by: score,price,name,provider,latency
--group-by <field>	Group by: provider,tier,status (new; defau
--format <type>	Output: table,compact,json,csv (new; defau
--no-color	Disable color output (new)
--no-emoji	Disable emoji/symbols (new)

4.2 Wide Terminal Mode (>= 120 columns)

ASCII Mockup — Wide Mode:

HelixCode Models					Updated 14:32
23 models 18 ✓ 3 ⌛ 2 0 ○					
MODEL	PROVIDER	S	SCORE	PRICE	CAPS
claude-opus-4-6	Anthropic	✓	9.4	\$15.0/1K	[?]⚡</>
gpt-4o	OpenAI	✓	9.1	\$5.0/1K	[?]⚡</>
gemini-2.5-pro	Google	✓	8.7	\$1.2/1K	[?]⚡</>
deepseek-chat	DeepSeek	✓	8.3	\$0.1/1K	⚡</>🧠
grok-3-fast-beta	xAI	✓	8.0	FREE	[?]⚡</>
llama-3.3-70b	Groq	✓	7.5	\$0.9/1K	⚡</>🔒
mistral-large	Mistral	✓	7.2	\$3.0/1K	[?]⚡</>
groq-llama-3.1-8b	Groq	✓	6.8	\$0.1/1K	⚡</>🔒
claude-sonnet-4-5	Anthropic	⌛	7.8	\$3.0/1K	[?]⚡</>
qwen-2.5-72b	Qwen	⌛	6.5	FREE	⚡</>🔒
openrouter-mixtral	OpenRouter	✗	4.2	\$0.6/1K	⚡🔒
ollama-llama3.2	Local	✓	6.0	FREE	⚡</>🔒

|| groq-llama-3.1-70b (12m) | 🚫 xai-free

4.4 Narrow Terminal Mode (< 80 columns)

ASCII Mockup — Narrow Mode:

HelixCode Models						14:32:05
23 models 18 OK 3 .. 2 RL 0 !						
#	MODEL	PROV.	ST	SC	PR	CAPS
1	claude-opus-4-6	Anthro.	OK	9.4	\$15	VSTCrR
2	gpt-4o	OpenAI	OK	9.1	\$5	VSTCr
3	gemini-2.5-pro	Google	OK	8.7	\$1	VSTCrR
4	deepseek-chat	DeepSk.	OK	8.3	\$0	STCr0
5	grok-3-fast-beta	xAI	OK	8.0	FREE	VSTCrR
6	llama-3.3-70b	Groq	OK	7.5	\$1	STCr0
7	mistral-large	Mistral	OK	7.2	\$3	VSTCr
8	groq-llama-3.1-8b	Groq	OK	6.8	\$0	STCr0
9	claude-sonnet-4-5	Anthro.	..	7.8	\$3	VSTCrR
10	qwen-2.5-72b	Qwen	..	6.5	FREE	STCr0
11	openrouter-mixtral	OpenR.	XX	4.2	\$1	ST0
12	ollama-llama3.2	Local	OK	6.0	FREE	STCr0
RL: groq-llama-3.1-70b(12m) QU: xai-free						

4.5 Grouped by Provider (Standard Mode)

ASCII Mockup — Grouped:

 HelixCode Models — Grouped by Provider						Updated 14:32:05
Anthropic • HEALTHY						
claude-opus-4-6	✓	9.4		\$15.0/1K	200.0K	
claude-sonnet-4-5		7.8		\$3.0/1K	200.0K	
OpenAI • HEALTHY						
gpt-4o	✓	9.1		\$5.0/1K	128.0K	
Groq • DEGRADED (⏸ llama-3.1-70b cooldown 12m)						
llama-3.3-70b	✓	7.5		\$0.9/1K	128.0K	
groq-llama-3.1-8b	✓	6.8		\$0.1/1K	128.0K	
llama-3.1-70b	⏸	7.1		\$0.6/1K	128.0K	
xAI QUOTA EXCEEDED						
grok-3-fast-beta	✓	8.0		FREE	131.0K	

grok-3-mini	x	6.2		FREE	131.0K	 
Local ● HEALTHY						
ollama-llama3.2	✓	6.0		FREE	8.0K	 
llamacpp-mistral-7b	✓	5.5		FREE	32.0K	 

4.6 Compact Mode (for scripting / piping)

ID	PROVIDER	STATUS	SCORE	PRICE/1K	CONTEXT	C
claude-opus-4-6	anthropic	verified	9.4	15.00	200000	v
gpt-4o	openai	verified	9.1	5.00	128000	v
gemini-2.5-pro	google	verified	8.7	1.25	100000	v
deepseek-chat	deepseek	verified	8.3	0.14	64000	s
grok-3-fast-beta	xai	verified	8.0	0.00	131072	v
llama-3.3-70b	groq	verified	7.5	0.90	128000	s
mistral-large	mistral	verified	7.2	3.00	128000	v
groq-llama-3.1-8b	groq	verified	6.8	0.05	128000	s
claude-sonnet-4-5	anthropic	pending	7.8	3.00	200000	v
qwen-2.5-72b	qwen	pending	6.5	0.00	128000	s
openrouter-mixtral	openrouter	failed	4.2	0.60	32000	s
ollama-llama3.2	local	verified	6.0	0.00	8000	st
llamacpp-mistral-7b	local	verified	5.5	0.00	32000	s

4.7 List Display Implementation

```
// internal/cli/ux/list_display.go

package ux

import (
    "fmt"
    "sort"
    "strings"
    "time"

    "github.com/fatih/color"
)

// ListDisplayOptions configures the model list rendering
type ListDisplayOptions struct {
    ProviderFilter    string
    VerifiedOnly      bool
    MaxPrice          float64
    MinScore          float64
    CapabilityFilter  string
    SortBy            string // "score", "price", "name",
    "provider", "latency"
    GroupBy           string // "provider", "tier", "status", ""
}
```



```

    Format          string // "table", "compact", "json", "csv"
    NoColor          bool
    NoEmoji          bool
    TerminalWidth    int
}

// ModelListRow represents a single row in the model list
type ModelListRow struct {
    Rank          int
    Model          *UnifiedModel
    Provider       *UnifiedProvider
    Verification   *VerificationResult
    Cooldown       *CooldownInfo
    ScoreBar       string
    PriceStr       string
    StatusBadge    string
    Capabilities   string
}

func RenderModelList(models []ModelListRow, opts
    *ListDisplayOptions) string {
    sym := NewSymbolSet(DetectTerminalCapabilities())
    if opts.NoEmoji {
        sym = NewSymbolSet(&TerminalCapabilities{IsWindowsCMD:
            true})
    }

    if opts.NoColor {
        color.NoColor = true
    }

    switch opts.Format {
    case "json":
        return renderJSON(models)
    case "csv":
        return renderCSV(models)
    case "compact":
        return renderCompact(models, sym, opts.TerminalWidth)
    default:
        return renderTable(models, sym, opts)
    }
}

func renderTable(rows []ModelListRow, sym *SymbolSet, opts
    *ListDisplayOptions) string {
    width := opts.TerminalWidth
    if width < 60 { width = 60 }

```

```

// Determine layout based on width
if width >= 120 {
    return renderWideTable(rows, sym, width)
} else if width >= 80 {
    return renderStandardTable(rows, sym, width)
}
return renderNarrowTable(rows, sym, width)
}

func renderWideTable(rows []ModelListRow, sym *SymbolSet, width
    int) string {
var b strings.Builder

// Header
header := fmt.Sprintf(" %s HelixCode – Available Models
    (verified by LLMsVerifier)", sym.Diamond)
b.WriteString(CHeader.Sprintf("%s\n", header))
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))

// Summary bar
verified := 0; pending := 0; failed := 0; cooldown := 0;
    offline := 0
for _, r := range rows {
    switch r.Verification.Status {
    case "verified": verified++
    case "pending": pending++
    case "failed": failed++
    }
    if r.Cooldown != nil { cooldown++ }
    if r.Provider.Status == "offline" { offline++ }
}
summary :=
    fmt.Sprintf(" %d models across providers | %s %d
        verified | %s %d pending | %s %d cooldown | %s %d
        offline",
        len(rows), sym.Verified, verified, sym.Pending, pending,
        sym.CoolDown, cooldown, sym.Offline, offline)
b.WriteString(CSubheader.Sprintf("%s\n", summary))
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))

// Column headers
b.WriteString(fmt.Sprintf(" %-24s | %-11s | %-6s | %-16s |
    %-10s | %-8s | %-25s\n",
    "MODEL NAME", "PROVIDER", "STATUS", "SCORE", "PRICE",
    "CONTEXT", "CAPABILITIES"))

```

```

b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))

// Rows
for _, r := range rows {
    tierPrefix := " "
    if r.Provider.Tier == 1 { tierPrefix = sym.StarFilled +
        " " }

    name := truncate(r.Model.DisplayName, 24)
    provider := truncate(r.Provider.DisplayName, 11)
    statusBadge := VerificationBadge(r.Verification.Status,
        sym).Render(sym, width)
    score := ScoreBadge(r.Verification.OverallScore, sym)
    price := PriceBadge(r.Model.CostPerInputToken,
        r.Model.CostPerOutputToken, sym, width)
    ctx := formatContextWindow(r.Model.ContextWindow)
    caps := CapabilityStripCompact(r.Model, sym)

    b.WriteString(fmt.Sprintf("%s%-24s | %-11s | %-6s | %-16s
        | %-10s | %-8s | %s\n",
            tierPrefix, name, provider, statusBadge, score,
            price, ctx, caps))
}

// Footer with cooldown alerts
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))
footerParts := []string{}
for _, r := range rows {
    if r.Cooldown != nil {
        badge := CooldownBadge(r.Cooldown.Reason,
            r.Cooldown.ResetTime, sym)
        footerParts = append(footerParts, badge.Render(sym,
            width))
    }
}
if len(footerParts) > 0 {
    b.WriteString(CAlertWarning.Sprintf(" %s\n",
        strings.Join(footerParts, " | ")))
}

return b.String()
}

func renderStandardTable(rows []ModelListRow, sym *SymbolSet,
    width int) string {

```

```

var b strings.Builder

b.WriteString(CHeader.Sprintf(" %s HelixCode Models\n",
    sym.Diamond))
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))

// Compact summary
counts := countStatuses(rows)
summary := fmt.Sprintf(" %d models | %d %s | %d %s | %d %s |
    %d %s",
    len(rows), counts.verified, sym.Verified,
    counts.pending, sym.Pending,
    counts.cooldown, sym.CoolDown, counts.offline,
    sym.Offline)
b.WriteString(CSubheader.Sprintf("%s\n", summary))
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))

// Headers
b.WriteString(fmt.Sprintf(" %-25s| %-10s| %-2s| %-5s| %-9s|
    %-15s\n",
    "MODEL", "PROVIDER", "S", "SCORE", "PRICE", "CAPS"))
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))

for _, r := range rows {
    name := truncate(r.Model.DisplayName, 25)
    provider := truncate(r.Provider.DisplayName, 10)
    status := VerificationBadge(r.Verification.Status,
        sym).Symbol
    scoreStr :=
        GetScoreColor(r.Verification.OverallScore).Printf("%.1f",
            r.Verification.OverallScore)
    priceStr := PriceBadge(r.Model.CostPerInputToken,
        r.Model.CostPerOutputToken, sym, width)
    caps := CapabilityStripCompact(r.Model, sym)

    b.WriteString(fmt.Sprintf(" %-25s| %-10s| %s| %s| %-9s|
        %s\n",
        name, provider, status, scoreStr, priceStr, caps))
}

// Cooldown footer
cooldownAlerts := getCooldownAlerts(rows, sym)
if len(cooldownAlerts) > 0 {
    b.WriteString(CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))
}

```

```

        b.WriteString(CAlertWarning.Sprintf(" %s\n",
            strings.Join(cooldownAlerts, " | ")))
    }

    return b.String()
}

func renderNarrowTable(rows []ModelListRow, sym *SymbolSet,
    width int) string {
    var b strings.Builder

    b.WriteString(CHeader.Sprintf(" HelixCode Models\n"))
    b.WriteString(CBorder.Sprintf("%s\n", strings.Repeat("-",
        width-1)))

    counts := countStatuses(rows)

    b.WriteString(fmt.Sprintf(" %d models | %d OK | %d .. |
        %d RL | %d !\n",
        len(rows), counts.verified, counts.pending,
        counts.cooldown, counts.offline))
    b.WriteString(CBorder.Sprintf("%s\n", strings.Repeat("-",
        width-1)))

    b.WriteString(fmt.Sprintf(" %-2s %-21s| %-7s| %-2s| %-2s|
        %-4s| %-6s\n",
        "#", "MODEL", "PROV.", "ST", "SC", "PR", "CAPS"))
    b.WriteString(CBorder.Sprintf("%s\n", strings.Repeat("-",
        width-1)))

    for i, r := range rows {
        name := truncate(r.Model.DisplayName, 21)
        prov := truncate(r.Provider.DisplayName, 7)
        status := VerificationBadge(r.Verification.Status,
            sym).Symbol
        score :=
            GetScoreColor(r.Verification.OverallScore).Printf("%.1f",
                r.Verification.OverallScore)
        price := PriceBadge(r.Model.CostPerInputToken,
            r.Model.CostPerOutputToken, sym, width)
        caps := CapabilityStripCompact(r.Model, sym)

        b.WriteString(fmt.Sprintf(" %-2d %-21s| %-7s| %s| %s|
            %-4s| %s\n",
            i+1, name, prov, status, score, price, caps))
    }

    return b.String()
}

```

```

}

func renderCompact(rows []ModelListRow, sym *SymbolSet, width
    int) string {
    var b strings.Builder
    b.WriteString(fmt.Sprintf("%-25s %-11s %-8s %-6s %-9s %-8s
        %s\n",
            "ID", "PROVIDER", "STATUS", "SCORE", "PRICE/1K",
            "CONTEXT", "CAPS"))
    for _, r := range rows {
        caps := strings.Join(r.Model.Capabilities, ",")
        price := fmt.Sprintf("%.2f",
            (r.Model.CostPerInputToken+r.Model.CostPerOutputToken)/
            2.0*1000)
        if price == "0.00" { price = "0.00" }
        b.WriteString(fmt.Sprintf("%-25s %-11s %-8s %-6.1f %-9s
            %-8d %s\n",
                r.Model.ID, r.Provider.DisplayName,
                r.Verification.Status,
                r.Verification.OverallScore, price,
                r.Model.ContextWindow, caps))
    }
    return b.String()
}

// --- Helper functions ---

func truncate(s string, maxLen int) string {
    if len(s) <= maxLen { return s }
    return s[:maxLen-1] + "..."
}

func formatContextWindow(n int) string {
    if n >= 1_000_000 {
        return fmt.Sprintf("%.1fM", float64(n)/1_000_000)
    }
    if n >= 1000 {
        return fmt.Sprintf("%.1fK", float64(n)/1000)
    }
    return fmt.Sprintf("%d", n)
}

type statusCounts struct {
    verified, pending, failed, cooldown, offline int
}

func countStatuses(rows []ModelListRow) statusCounts {
    c := statusCounts{}

```

```

for _, r := range rows {
    switch r.Verification.Status {
    case "verified": c.verified++
    case "pending": c.pending++
    case "failed": c.failed++
    }
    if r.Cooldown != nil { c.cooldown++ }
    if r.Provider.Status == "offline" { c.offline++ }
}
return c
}

func getCooldownAlerts(rows []ModelListRow, sym *SymbolSet)
    []string {
    alerts := []string{}
    for _, r := range rows {
        if r.Cooldown != nil {
            badge := CooldownBadge(r.Cooldown.Reason,
                r.Cooldown.ResetTime, sym)
            alerts = append(alerts, badge.Render(sym, 80))
        }
    }
    return alerts
}

```

5. Model Detail Display (- -model-info <id>)

5.1 CLI Flags to Add

```

--model-info <id>                Show detailed information for a model (new
--model-info-format <type>        Output: rich,json,yaml (new; default: ri

```

5.2 Rich Detail View (>= 100 columns)

ASCII Mockup:

 HelixCode – Model Details									
claude-opus-4-6					Anthropic • HEALTHY				
Status	✓ VERIFIED				Overall Score	9.4			
Tier	★★★★★ Premium				Code Capability	9.6			
Latency	234ms 🚀 Fast				Responsiveness	8.9			
Verified	2026-04-30 08:15 UTC				Reliability	9.2			
					Feature Richness	8.5			
					Value Prop.	7.8			

Context & Token Limits

Context Window

200,000 tokens

Architecture

transformer

Max Output Tokens

4,096

Release Date

2026-0

Pricing (per 1K tokens)

Input

\$15.00

Output

\$75.00

Cached Input

\$7.50

Expensive

Capabilities

✓ Vision

✓ Reasoning

✓ Open Source

✓ Streaming

× Audio

× Deprecated

✓ Tool Use

× Video

✓ Function Calling

✓ Code Generat

× Embeddings

✓ JSON Mode

Verification Dimensions

Model Exists

✓ PASS

Not Overloaded

✓ PASS

Code Generation

✓ PASS

Code Optimization

✓ PASS

Documentation Gen.

✓ PASS

Security Assessment

✓ PASS

Responsive

✓ PASS

Supports Tools

✓ PASS

Code Debugging

✓ PASS

Test Generation

✓ PASS

Architecture

✓ PASS

Pattern Recog.

✓ PASS

Rate Limits

Type

Limit

Used

Remaining

Reset In

Requests/min

100

23

77

14:42:05

Tokens/min

50,000

12,340

37,660

14:42:05

Provider Health

Status: ● HEALTHY

Uptime: 99.97%

Last Check: 14:32:01

P95 Late

Alternative Models

If unavailable, HelixCode will auto-select:

1. claude-sonnet-4-6 (Anthropic) — Score: 9.0 — Price: \$3.00/1K

2. gpt-4o (OpenAI) — Score: 9.1 — Price: \$5.00/1K

3. gemini-2.5-pro (Google) — Score: 8.7 — Price: \$1.25/1K

Tags

coding, reasoning, long-context, enterprise, premium

Languages

en, es, fr, de, it, pt, zh, ja, ko, ar, hi, ru

5.3 Compact Detail View (60-99 columns)

ASCII Mockup:

HelixCode — Model Details

claude-opus-4-6

Anthropic • HEALTHY

Status: ✓ VERIFIED

Score: 9.4 (Excellent)

Tier: ★★★★★ Premium

Latency: 234ms Fast

Verified: 2026-04-30

Context: 200K tokens MaxOut: 4,096 tokens

— Pricing —

Input: \$15.00/1K Output: \$75.00/1K Cached: \$7.50/1K

[] Expensive

— Capabilities —

✓ vision ✓ streaming ✓ tools ✓ code ✓ reasoning

× audio × video × embeddings

— Verification —

exists ✓ responsive ✓ overloaded × tools ✓ code ✓ debug ✓

optimize ✓ test ✓ docs ✓ architecture ✓ security ✓ patterns ✓

— Rate Limits —

req/min: 100 limit, 23 used, 77 remaining (resets 14:42:05)

tok/min: 50K limit, 12K used, 38K remaining (resets 14:42:05)

— Fallbacks —

1. claude-sonnet-4-6 (9.0, \$3.00/1K)

2. gpt-4o (9.1, \$5.00/1K)

3. gemini-2.5-pro (8.7, \$1.25/1K)

5.4 Narrow Detail View (< 60 columns)

ASCII Mockup:

Model: claude-opus-4-6

Provider: Anthropic [+]

Status: VERIFIED [OK]

Score: 9.4

Latency: 234ms [FAST]

Context: 200K / MaxOut: 4096

Price In: \$15.00/1K Out: \$75.00/1K

[] Expensive

Capabilities:

OK vision streaming tools code reasoning

NO audio video embeddings

Verification:

OK: exists responsive tools code debug

optimize test docs arch security

Rate Limits:

```
req/min: 100, 23 used, 77 left  
tok/min: 50K, 12K used, 38K left  
Reset: 14:42:05
```

Fallbacks:

1. claude-sonnet-4-6 (9.0, \$3)
2. gpt-4o (9.1, \$5)
3. gemini-2.5-pro (8.7, \$1.2)

5.5 Detail View Implementation

```
// internal/cli/ux/detail_display.go
```

```
package ux
```

```
import (  
    "fmt"  
    "strings"  
    "time"  
)
```

```
type DetailDisplayOptions struct {  
    Format          string // "rich", "json", "yaml"  
    NoColor         bool  
    NoEmoji         bool  
    TerminalWidth  int  
}
```

```
func RenderModelDetail(model *UnifiedModel, provider  
    *UnifiedProvider,  
    verification *VerificationResult, limits *RateLimitStatus,  
    cooldown *CooldownInfo, alternatives []*UnifiedModel,  
    opts *DetailDisplayOptions) string {  
  
    sym := NewSymbolSet(DetectTerminalCapabilities())  
    if opts.NoEmoji {  
        sym = NewSymbolSet(&TerminalCapabilities{IsWindowsCMD:  
            true})  
    }  
    if opts.NoColor {  
        color.NoColor = true  
    }  
  
    switch opts.Format {  
    case "json":  
        return renderDetailJSON(model, provider, verification,  
            limits, cooldown, alternatives)  
    case "yaml":
```

```

        return renderDetailYAML(model, provider, verification,
                                limits, cooldown, alternatives)
    default:
        return renderDetailRich(model, provider, verification,
                                limits, cooldown, alternatives, sym, opts.TerminalWidth)
    }
}

```

```

func renderDetailRich(m *UnifiedModel, p *UnifiedProvider, v
    *VerificationResult,
    limits *RateLimitStatus, cd *CooldownInfo, alts
    []*UnifiedModel,
    sym *SymbolSet, width int) string {

    var b strings.Builder

    if width >= 100 {
        b.WriteString(renderDetailHeaderWide(m, p, sym, width))
        b.WriteString(renderScorePanelWide(v, sym, width))
        b.WriteString(renderContextPanelWide(m, sym, width))
        b.WriteString(renderPricingPanelWide(m, sym, width))
        b.WriteString(renderCapabilitiesPanelWide(m, v, sym,
            width))
        b.WriteString(renderVerificationPanelWide(v, sym, width))
        if limits != nil {
            b.WriteString(renderRateLimitPanelWide(limits, sym,
                width))
        }
        b.WriteString(renderProviderHealthPanelWide(p, sym,
            width))
        if len(alts) > 0 {
            b.WriteString(renderAlternativesPanelWide(alts, sym,
                width))
        }
    } else if width >= 60 {
        b.WriteString(renderDetailHeaderCompact(m, p, sym,
            width))
        b.WriteString(renderScorePanelCompact(v, sym, width))
        b.WriteString(renderContextPanelCompact(m, sym, width))
        b.WriteString(renderPricingPanelCompact(m, sym, width))
        b.WriteString(renderCapabilitiesPanelCompact(m, v, sym,
            width))
        b.WriteString(renderVerificationPanelCompact(v, sym,
            width))
        if limits != nil {
            b.WriteString(renderRateLimitPanelCompact(limits,
                sym, width))
        }
    }
}

```

```

        if len(alts) > 0 {
            b.WriteString(renderAlternativesPanelCompact(alts,
                sym, width))
        }
    } else {
        b.WriteString(renderDetailHeaderNarrow(m, p, sym, width))
        b.WriteString(renderScorePanelNarrow(v, sym, width))
        b.WriteString(renderPricingPanelNarrow(m, sym, width))
        b.WriteString(renderCapabilitiesPanelNarrow(m, v, sym,
            width))
        b.WriteString(renderVerificationPanelNarrow(v, sym,
            width))
        if len(alts) > 0 {
            b.WriteString(renderAlternativesPanelNarrow(alts,
                sym, width))
        }
    }

    return b.String()
}

```

// Wide header

```

func renderDetailHeaderWide(m *UnifiedModel, p *UnifiedProvider,
    sym *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CHeader.Sprintf(" %s HelixCode — Model
        Details\n", sym.Diamond))
    b.WriteString(CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))
    b.WriteString("\n")

    healthBadge := ProviderHealthBadge(p.Status,
        sym).Render(sym, width)
    nameLine := fmt.Sprintf(" %-50s %s %s", m.DisplayName,
        p.DisplayName, healthBadge)
    b.WriteString(CAccent.Sprintf("%s\n", nameLine))
    b.WriteString(CBorder.Sprintf(" %s\n", strings.Repeat("=",
        width-5)))
    b.WriteString("\n")

    return b.String()
}

```

// Score panel with bar visualization

```

func renderScorePanelWide(v *VerificationResult, sym *SymbolSet,
    width int) string {
    var b strings.Builder

```

```

overallLabel := CLabel.Sprint(" Overall Score ")
overallBar := ScoreBadge(v.OverallScore, sym)
overallDesc := ""
switch {
case v.OverallScore >= 9.0: overallDesc =
    CScoreExcellent.Sprint("(Excellent)")
case v.OverallScore >= 7.0: overallDesc =
    CScoreGood.Sprint("(Good)")
case v.OverallScore >= 5.0: overallDesc =
    CScoreAverage.Sprint("(Average)")
case v.OverallScore >= 3.0: overallDesc =
    CScorePoor.Sprint("(Poor)")
default: overallDesc = CScoreBad.Sprint("(Bad)")
}

b.WriteString(fmt.Sprintf(" %s %s %s\n", overallLabel,
    overallBar, overallDesc))
b.WriteString(fmt.Sprintf(" %s %s\n", CLabel.Sprint(" Code
    Capability "), ScoreBadge(v.CodeCapabilityScore, sym)))
b.WriteString(fmt.Sprintf(" %s %s\n", CLabel.Sprint("
    Responsiveness "), ScoreBadge(v.ResponsivenessScore,
    sym)))
b.WriteString(fmt.Sprintf(" %s %s\n", CLabel.Sprint("
    Reliability "), ScoreBadge(v.ReliabilityScore, sym)))
b.WriteString(fmt.Sprintf(" %s %s\n", CLabel.Sprint("
    Feature Richness"), ScoreBadge(v.FeatureRichnessScore,
    sym)))
b.WriteString(fmt.Sprintf(" %s %s\n",
    CLabel.Sprint(" Value Prop. "),
    ScoreBadge(v.ValuePropositionScore, sym)))
b.WriteString("\n")

return b.String()
}

// Pricing panel with visual bar
func renderPricingPanelWide(m *UnifiedModel, sym *SymbolSet,
    width int) string {
var b strings.Builder

b.WriteString(CSubheader.Sprintf(" — Pricing (per 1K
    tokens) %s\n", strings.Repeat("-", width-35)))

inPrice := m.CostPerInputToken * 1000
outPrice := m.CostPerOutputToken * 1000
cachedPrice := inPrice * 0.5 // typical cached rate

```

```

priceLine := fmt.Sprintf("  Input   %%6.2f    Output
                        %%6.2f    Cached Input   %%6.2f",
                        sym.Dollar, inPrice, sym.Dollar, outPrice, sym.Dollar,
                        cachedPrice)
b.WriteString(CValue.Sprintf("%s\n", priceLine))

// Price intensity bar
avgPrice := (inPrice + outPrice) / 2.0
barWidth := width - 6
filled := int((avgPrice / 20.0) * float64(barWidth)) // max
                        $20 = full bar
if filled > barWidth { filled = barWidth }
if filled < 0 { filled = 0 }

barColor := CBarGood
if avgPrice > 2.0 { barColor = CBarAverage }
if avgPrice > 5.0 { barColor = CBarPoor }

barStr := barColor.Sprintf("%s",
    strings.Repeat(sym.ProgressFull, filled)) +
    CBarEmpty.Sprintf("%s",
    strings.Repeat(sym.ProgressEmpty, barWidth-filled))
b.WriteString(fmt.Sprintf("  %s\n", barStr))

priceLabel := ""
switch {
case avgPrice == 0: priceLabel = CPriceFree.Sprint("FREE")
case avgPrice < 0.5: priceLabel = CPriceCheap.Sprint("Cheap")
case avgPrice < 2.0: priceLabel =
    CPriceModerate.Sprint("Moderate")
default: priceLabel = CPriceExpensive.Sprint("Expensive")
}
b.WriteString(fmt.Sprintf("  %s %s\n", sym.ArrowUp,
    priceLabel))
b.WriteString("\n")

return b.String()
}

// Capabilities grid
func renderCapabilitiesPanelWide(m *UnifiedModel, v
    *VerificationResult, sym *SymbolSet, width int) string {
var b strings.Builder
b.WriteString(CSubheader.Sprintf("  — Capabilities %s\n",
    strings.Repeat("-", width-22)))

caps := []struct{
    label string

```

```

    val bool
    sym string
}{
    {"Vision", m.SupportsVision, sym.Vision},
    {"Streaming", m.SupportsStreaming, sym.Streaming},
    {"Tool Use", m.SupportsTools, sym.Tools},
    {"Code Generation", v.SupportsCodeGeneration, sym.Code},
    {"Reasoning", v.SupportsReasoning, sym.Reasoning},
    {"Audio", m.SupportsAudio, sym.Audio},
    {"Video", m.SupportsVideo, sym.Video},
    {"Embeddings", v.SupportsEmbeddings, sym.Embeddings},
    {"Open Source", m.OpenSource, sym.OpenSource},
    {"Deprecated", m.Deprecated, "△"},
    {"Function Calling", m.SupportsFunctions, sym.Tools},
    {"JSON Mode", v.SupportsJSONMode, "{ }"},
}

// 4 columns
colWidth := (width - 8) / 4
for i := 0; i < len(caps); i += 4 {
    lineParts := []string{}
    for j := 0; j < 4 && i+j < len(caps); j++ {
        c := caps[i+j]
        status := CFailed.Sprintf("x")
        if c.val { status = CVerified.Sprintf("✓") }
        part := fmt.Sprintf(" %s %-18s", status, c.label)
        lineParts = append(lineParts, part)
    }
    b.WriteString(strings.Join(lineParts, "") + "\n")
}
b.WriteString("\n")

return b.String()
}

// Verification dimensions
func renderVerificationPanelWide(v *VerificationResult, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CSubheader.Sprintf(" — Verification
        Dimensions %s\n", strings.Repeat("-", width-33)))

    checks := []struct{ name string; pass bool }{
        {"Model Exists", v.ModelExists},
        {"Responsive", v.Responsive},
        {"Not Overloaded", !v.Overloaded},
        {"Supports Tools", v.SupportsToolUse},
        {"Code Generation", v.SupportsCodeGeneration},
    }

```

```

        {"Code Debugging", v.CodeDebugging},
        {"Code Optimization", v.CodeOptimization},
        {"Test Generation", v.TestGeneration},
        {"Documentation Gen.", v.DocumentationGeneration},
        {"Architecture", v.ArchitectureDesign},
        {"Security Assessment", v.SecurityAssessment},
        {"Pattern Recognition", v.PatternRecognition},
    }

    // 2 columns
    mid := (len(checks) + 1) / 2
    for i := 0; i < mid; i++ {
        left := checks[i]
        leftStr := fmt.Sprintf(" %s %-22s",
            passFail(left.pass), left.name)

        rightStr := ""
        if i+mid < len(checks) {
            right := checks[i+mid]
            rightStr = fmt.Sprintf(" %s %-22s",
                passFail(right.pass), right.name)
        }
        b.WriteString(leftStr + rightStr + "\n")
    }
    b.WriteString("\n")

    return b.String()
}

func passFail(p bool) string {
    if p { return CVerified.Sprint("✓") }
    return CFailed.Sprint("×")
}

// Rate limit panel
func renderRateLimitPanelWide(limits *RateLimitStatus, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CSubheader.Sprintf(" — Rate Limits %s\n",
        strings.Repeat("—", width-21)))
    b.WriteString(fmt.Sprintf(" %-16s %-8s %-8s %-12s %s\n",
        "Type", "Limit", "Used", "Remaining", "Reset In"))

    for _, l := range limits.Limits {
        resetStr := formatDuration(time.Until(l.ResetTime))
        b.WriteString(fmt.Sprintf(" %-16s %-8d %-8d %-12d %s\n",
            l.Type, l.Limit, l.Used, l.Remaining, resetStr))
    }
}

```



```

    b.WriteString("\n")
    return b.String()
}

// Provider health
func renderProviderHealthPanelWide(p *UnifiedProvider, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CSubheader.Sprintf(" — Provider Health
        %s\n", strings.Repeat("-", width-25)))

    healthBadge := ProviderHealthBadge(p.Status,
        sym).Render(sym, width)
    uptimeStr := fmt.Sprintf("%.2f%%", p.UptimePct)
    lastCheck := p.LastHealthCheck.Format("15:04:05")

    b.WriteString(fmt.Sprintf(" Status: %s      Uptime: %s
        Last Check: %s      P95 Latency: %s\n",
        healthBadge, CValue.Sprint(uptimeStr),
        CValue.Sprint(lastCheck),
        CValue.Sprint(formatLatency(p.Latency))))
    b.WriteString("\n")
    return b.String()
}

// Alternatives panel
func renderAlternativesPanelWide(alts []*UnifiedModel, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CSubheader.Sprintf(" — Alternative Models
        %s\n", strings.Repeat("-", width-28)))

    b.WriteString(CAlertInfo.Sprintf(" If unavailable,
        HelixCode will auto-select:\n"))

    for i, alt := range alts {
        if i >= 5 { break }
        price := (alt.CostPerInputToken +
            alt.CostPerOutputToken) / 2.0 * 1000
        b.WriteString(fmt.Sprintf("      %d. %s (%s) — Score: %s —
            Price: %s%.2f/1K\n",
            i+1, alt.DisplayName, alt.Provider,
            GetScoreColor(alt.Score).Sprintf("%.1f", alt.Score),
            sym.Dollar, price))
    }
    b.WriteString("\n")
    return b.String()
}

```

```

}

func formatDuration(d time.Duration) string {
    if d < 0 { return "now" }
    if d < time.Minute { return fmt.Sprintf("%ds",
        int(d.Seconds())) }
    if d < time.Hour { return fmt.Sprintf("%dm",
        int(d.Minutes())) }
    return fmt.Sprintf("%dh%dm", int(d.Hours()), int(d.Minutes())
        %60)
}

func formatLatency(d time.Duration) string {
    if d < time.Millisecond { return fmt.Sprintf("%dμs",
        d.Microseconds()) }
    if d < time.Second { return fmt.Sprintf("%dms",
        d.Milliseconds()) }
    return fmt.Sprintf("%.1fs", d.Seconds())
}

```

6. Interactive Model Selection

6.1 User Flow

```

$ ./cli
helix> models

```


HelixCode – Model Selector

Model List	Preview
1 ★ claude-opus-4-6 2 gpt-4o 3 gemini-2.5-pro 4 deepseek-chat 5 grok-3-fast-beta 6 llama-3.3-70b 7 mistral-large 8 groq-llama-3.1-8b 9 claude-sonnet-4-5 10 qwen-2.5-72b 11 openrouter-mixtral 12 ollama-llama3.2 13 llamacpp-mistral-7b	claude-opus-4-6 Anthropic • HEALTHY Score: 9.4 (Excellent) Latency: 234ms 🚀 Fast Price: \$15.00/1K (input) / \$75.00 Context: 200K tokens Max Out: 4,096 tokens Capabilities: ✓ vision ✓ streaming ✓ tools ✓ code ✓ reasoning × audio × video × embeddings Verification: ✓ VERIFIED on 2026-04-30 08:15 U [Use this model] [View full deta

```
Filter: [all]  Sort: [score▼]  Group: [none]
[f]ilter  [s]ort  [g]roup  [r]efresh  [q]uit  [↑↓]navigate  [Enter]select
Status: ● 18 healthy  ● 1 degraded  ■ 2 cooldown
```

6.2 TUI Architecture (using tview)

```
// internal/cli/tui/model_selector.go

package tui

import (
    "context"
    "fmt"
    "strings"
    "time"

    "github.com/fatih/color"
    "github.com/rivo/tview"
)

// ModelSelectorApp is the interactive model selection TUI
type ModelSelectorApp struct {
    app            *tview.Application
    modelList      *tview.List
    previewPane    *tview.TextView
    statusBar      *tview.TextView
    filterInput    *tview.InputField

    models         []*ModelListRow
    filtered        []*ModelListRow
    selectedIndex  int

    sym            *ux.SymbolSet
    refreshTicker  *time.Ticker
    cancelFunc     context.CancelFunc

    // Callback when user selects a model
    onSelect      func(modelID string)
}

func NewModelSelectorApp(models []*ux.ModelListRow, sym
    *ux.SymbolSet) *ModelSelectorApp {
    m := &ModelSelectorApp{
        app:      tview.NewApplication(),
        models:   models,
    }
```

```

        filtered: models,
        sym:      sym,
    }
    m.buildUI()
    return m
}

func (m *ModelSelectorApp) buildUI() {
    // Model list (left pane)
    m.modelList = tview.NewList()
    m.modelList.SetBorder(true)
    m.modelList.SetTitle(" Model List ")
    m.modelList.SetMainTextColor(tview.ColorWhite)
    m.modelList.SetSecondaryTextColor(tview.ColorDarkGray)
    m.modelList.SetSelectedBackgroundColor(tview.ColorDarkCyan)

    m.populateModelList()

    m.modelList.SetSelectedFunc(func(idx int, mainText string,
        secondaryText string, shortcut rune) {
        if idx >= 0 && idx < len(m.filtered) {
            m.onSelect(m.filtered[idx].Model.ID)
            m.app.Stop()
        }
    })

    m.modelList.SetChangedFunc(func(idx int, mainText string,
        secondaryText string, shortcut rune) {
        if idx >= 0 && idx < len(m.filtered) {
            m.updatePreview(m.filtered[idx])
        }
    })

    // Preview pane (right pane)
    m.previewPane = tview.NewTextView()
    m.previewPane.SetBorder(true)
    m.previewPane.SetTitle(" Preview ")
    m.previewPane.SetDynamicColors(true)
    m.previewPane.SetScrollable(true)

    // Status bar (bottom)
    m.statusBar = tview.NewTextView()
    m.statusBar.SetDynamicColors(true)
    m.statusBar.SetTextAlign(tview.AlignLeft)

    // Filter input
    m.filterInput = tview.NewInputField()
    m.filterInput.SetLabel("Filter: ")

```

```

m.filterInput.SetFieldBackgroundColor(tview.ColorBlack)
m.filterInput.SetDoneFunc(func(key tcell.Key) {
    m.applyFilter(m.filterInput.GetText())
})

// Layout
mainFlex := tview.NewFlex().
    AddItem(m.modelList, 0, 1, true).
    AddItem(m.previewPane, 0, 2, false)

fullLayout := tview.NewFlex().SetDirection(tview.FlexRow).
    AddItem(mainFlex, 0, 1, true).
    AddItem(m.filterInput, 1, 0, false).
    AddItem(m.statusBar, 1, 0, false)

m.app.SetRoot(fullLayout, true)

// Key bindings
m.app.SetInputCapture(func(event *tcell.EventKey)
    *tcell.EventKey {
        switch event.Rune() {
        case 'q', 'Q':
            m.app.Stop()
            return nil
        case 'r', 'R':
            m.triggerRefresh()
            return nil
        case 'f', 'F':
            m.app.SetFocus(m.filterInput)
            return nil
        }
        return event
    })

// Initial preview
if len(m.filtered) > 0 {
    m.updatePreview(m.filtered[0])
}
m.updateStatusBar()
}

func (m *ModelSelectorApp) populateModelList() {
    m.modelList.Clear()
    for i, r := range m.filtered {
        tierPrefix := " "
        if r.Provider.Tier == 1 { tierPrefix = "★ " }
    }
}

```

```

mainText := fmt.Sprintf("%s%s", tierPrefix,
    r.Model.DisplayName)

// Secondary text with status, score, price
statusBadge :=
    ux.VerificationBadge(r.Verification.Status, m.sym).Symbol
scoreStr := fmt.Sprintf("%.1f",
    r.Verification.OverallScore)
priceStr := ux.PriceBadge(r.Model.CostPerInputToken,
    r.Model.CostPerOutputToken, m.sym, 80)

secondaryText := fmt.Sprintf("  %s %s  %s  %s  %s",
    statusBadge, scoreStr, priceStr,
    r.Provider.DisplayName,
    ux.CapabilityStripCompact(r.Model, m.sym))

m.modelList.AddItem(mainText, secondaryText, rune('0'+
    ((i+1)%10)), nil)
}
}

func (m *ModelSelectorApp) updatePreview(r *ux.ModelListRow) {
    // Render compact detail view into preview pane
    detail := ux.RenderCompactDetailForPreview(r, m.sym)
    m.previewPane.SetText(detail)
}

func (m *ModelSelectorApp) updateStatusBar() {
    counts := ux.CountStatuses(m.filtered)
    text := fmt.Sprintf(
        " [green]● %d healthy[-]  [yellow]● %d degraded[-]
        [red]■ %d cooldown[-]  [gray]○ %d offline[-]  |  [blue]
        [f]ilter [s]ort [g]roup [r]efresh [q]uit[-]",
        counts.Healthy, counts.Degraded, counts.Cooldown,
        counts.Offline)
    m.statusBar.SetText(text)
}

func (m *ModelSelectorApp) applyFilter(query string) {
    query = strings.ToLower(strings.TrimSpace(query))
    if query == "" {
        m.filtered = m.models
    } else {
        m.filtered = []*ux.ModelListRow{}
        for _, r := range m.models {
            if
                strings.Contains(strings.ToLower(r.Model.DisplayName),
                    query) ||

```

```

        strings.Contains(strings.ToLower(r.Provider.DisplayName),
        query) ||
            strings.Contains(strings.ToLower(r.Model.ID),
        query) ||

        strings.Contains(strings.ToLower(strings.Join(r.Model.Capabilities
        " ")), query) {
            m.filtered = append(m.filtered, r)
        }
    }
}
m.populateModelList()
if len(m.filtered) > 0 {
    m.updatePreview(m.filtered[0])
}
m.updateStatusBar()
}

func (m *ModelSelectorApp) triggerRefresh() {
    // Signal to background goroutine to refresh data
    // Updates models slice, re-applies filter, repopulates list
}

func (m *ModelSelectorApp) Run() error {
    return m.app.Run()
}

func (m *ModelSelectorApp) GetSelectedModel() string {
    if m.selectedIndex >= 0 && m.selectedIndex < len(m.filtered)
    {
        return m.filtered[m.selectedIndex].Model.ID
    }
    return ""
}

```

6.3 Fallback: Numbered Menu Mode (for terminals without tview support)

If `tview` cannot initialize (e.g., non-TTY, CI environment, minimal terminal), fall back to a numbered interactive menu:

```

// internal/cli/ux/interactive_fallback.go

func RenderNumberedModelMenu(models []*ux.ModelListRow, sym
    *ux.SymbolSet, width int) string {
    var b strings.Builder

    b.WriteString(ux.CHeader.Sprintf("%s HelixCode – Model
    Selector\n", sym.Diamond))

```

```

b.WriteString(ux.CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))
b.WriteString("\n")

for i, r := range models {
    status := ux.VerificationBadge(r.Verification.Status,
        sym).Symbol
    score := fmt.Sprintf("%.1f", r.Verification.OverallScore)
    price := ux.PriceBadge(r.Model.CostPerInputToken,
        r.Model.CostPerOutputToken, sym, width)

    b.WriteString(fmt.Sprintf(" [%s%d%s] %-30s %s %s %s
        %s %s\n",
            ux.CAccent.Sprint(), i+1, "[white]",
            ux.Truncate(r.Model.DisplayName, 30),
            status, score, price, r.Provider.DisplayName,
            ux.CapabilityStripCompact(r.Model, sym)))
}

b.WriteString("\n")

    b.WriteString(ux.CSubheader.Sprint(" Enter number to
        select, [f]ilter, [s]ort, [q]uit: "))

return b.String()
}

func RunNumberedInteractiveSelector(models []*ux.ModelListRow,
    sym *ux.SymbolSet) (string, error) {
    reader := bufio.NewReader(os.Stdin)
    current := models

    for {
        width, _, _ := term.GetSize(int(os.Stdout.Fd()))
        if width < 60 { width = 80 }

        fmt.Print(RenderNumberedModelMenu(current, sym, width))

        fmt.Print("\n> ")
        input, err := reader.ReadString('\n')
        if err != nil { return "", err }
        input = strings.TrimSpace(strings.ToLower(input))

        switch input {
        case "q", "quit", "exit":
            return "", fmt.Errorf("selection cancelled")
        case "f", "filter":

```



```

        fmt.Print("Filter by name/provider/capability: ")
        filter, _ := reader.ReadString('\n')
        current = applyFilterString(models,
strings.TrimSpace(filter))
    case "s", "sort":
        fmt.Print("Sort by [score/price/name/latency]: ")
        sortBy, _ := reader.ReadString('\n')
        current = applySort(current,
strings.TrimSpace(sortBy))
    default:
        // Try to parse as number
        if num, err := strconv.Atoi(input); err == nil &&
num > 0 && num <= len(current) {
            return current[num-1].Model.ID, nil
        }

        fmt.Println("Invalid selection. Please enter a number or
command.")
    }
}
}

```

7. Notification / Alert UX

7.1 Alert Types & Rendering

```

// internal/cli/ux/alerts.go

type AlertLevel int

const (
    AlertInfo AlertLevel = iota
    AlertWarning
    AlertError
    AlertSuccess
)

type Alert struct {
    Level      AlertLevel
    Title      string
    Message    string
    ModelID    string
    Provider   string
    SuggestedAlternative string
    Timestamp time.Time
}

```

```

func (a *Alert) Render(sym *SymbolSet, width int) string {
    var b strings.Builder

    icon := ""
    titleColor := ux.CAlertInfo
    borderColor := ux.CBorder

    switch a.Level {
    case AlertInfo:
        icon = sym.Bullet
        titleColor = ux.CAlertInfo
    case AlertWarning:
        icon = sym.Degraded
        titleColor = ux.CAlertWarning
    case AlertError:
        icon = sym.Failed
        titleColor = ux.CAlertError
    case AlertSuccess:
        icon = sym.Verified
        titleColor = ux.CAlertSuccess
    }

    // Alert box
    b.WriteString(borderColor.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))
    b.WriteString(fmt.Sprintf("  %s %s\n", icon,
        titleColor.Sprint(a.Title)))
    b.WriteString(fmt.Sprintf("  %s\n", a.Message))

    if a.SuggestedAlternative != "" {
        b.WriteString(fmt.Sprintf("  %s Suggested alternative:
        %s\n", sym.ArrowRight, a.SuggestedAlternative))
    }

    b.WriteString(borderColor.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))

    return b.String()
}

```

7.2 Alert Scenarios

A. Model Enters Cooldown During Session:

■ COOLDOWN ALERT

Model "groq-llama-3.1-70b" has entered rate-limited cooldown.
Reason: Rate limit exceeded (150/100 req/min). Reset in 12m 34s.

→ Suggested alternative: llama-3.3-70b (Groq) – Score: 7.5

B. Provider Becomes Unavailable:

✗ PROVIDER OFFLINE
Provider "xAI" is now OFFLINE. All xAI models are unavailable.
Affected models: grok-3-fast-beta, grok-3-mini, grok-3
→ Suggested alternatives: gpt-4o (OpenAI), gemini-2.5-pro (Google)

C. Better Alternative Discovered:

✓ BETTER MODEL AVAILABLE
A higher-scoring alternative to "gpt-4o" is now available:
claude-opus-4-6 – Score: 9.4 (vs your current 9.1)
Same price range. Better code capability and reasoning.
→ Switch? [Y/n]

D. LLMsVerifier Connection Lost:

⚠ VERIFIER CONNECTION LOST
Lost connection to LLMsVerifier at http://localhost:8081.
Model verification data may be stale. Last update: 14:32:05.
→ Auto-retry in 30s... [r]etry now [c]ontinue with cached data

7.3 Auto-Suggest on Selection of Unavailable Model

\$./cli --prompt "Hello" --model groq-llama-3.1-70b

⚠ SELECTED MODEL UNAVAILABLE
"groq-llama-3.1-70b" is currently in cooldown (rate-limited).

Auto-switch to best available alternative?
[1] llama-3.3-70b (Groq) – Score: 7.5 – \$0.90/1K [RECOMMENDED]
[2] claude-sonnet-4-5 (Anthropic) – Score: 7.8 – \$3.00/1K
[3] gpt-4o (OpenAI) – Score: 9.1 – \$5.00/1K
[4] deepseek-chat (DeepSeek) – Score: 8.3 – \$0.14/1K
[5] Cancel and exit

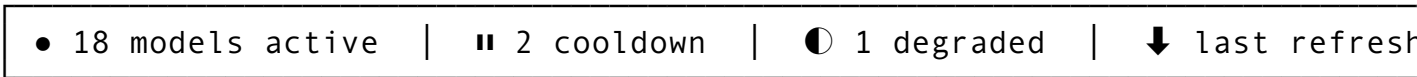
Select [1-5] or press Enter for default [1]:

8. Real-time Updates Display

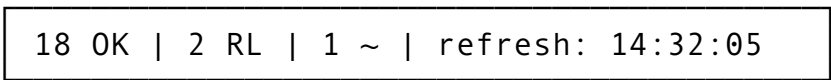
8.1 Status Bar Design

A persistent status bar at the bottom of all interactive model views:

Wide Status Bar:

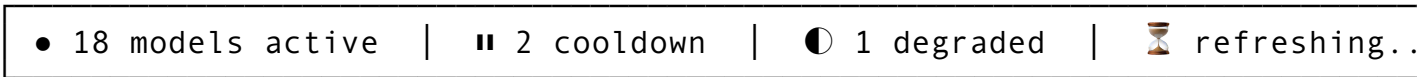


Narrow Status Bar:

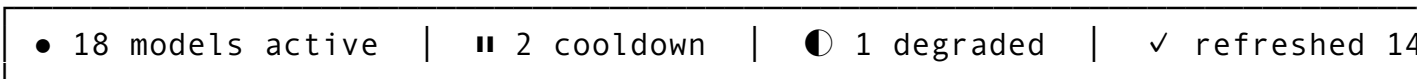


8.2 Refresh Indicator

When refresh is in progress:



When refresh completes:



8.3 Non-Clutter Update Strategy

For non-interactive mode (standard `--list-models` output), updates are NOT shown in real-time. The output is a snapshot.

For interactive TUI mode: - Background goroutine polls `LLMsVerifier` every 30-60 seconds - Only shows update when state CHANGES (model status, score, cooldown) - Updates use color flash or brief banner (disappears after 5s) - No scrolling disruption — updates appear in status bar only

```
// internal/cli/ux/status_bar.go
```

```
type StatusBar struct {
    sym          *ux.SymbolSet
    totalModels  int
    activeModels int
    cooldownCount int
    degradedCount int
    offlineCount int
    lastRefresh  time.Time
}
```

```

    isRefreshing    bool
    width           int
}

func (sb *StatusBar) Render() string {
    var b strings.Builder

    if sb.width >= 100 {
        b.WriteString(fmt.Sprintf(" %s %d models active %s %s
%d cooldown %s %s %d degraded",
            sb.sym.Healthy, sb.activeModels,
            sb.sym.SepVertical,
            sb.sym.CoolDown, sb.cooldownCount,
            sb.sym.SepVertical,
            sb.sym.Degraded, sb.degradedCount))
        if sb.isRefreshing {
            b.WriteString(fmt.Sprintf(" %s %s refreshing...",
                sb.sym.SepVertical, sb.sym.Pending))
        } else {
            b.WriteString(fmt.Sprintf(" %s %s last refresh:
%s",
                sb.sym.SepVertical, sb.sym.Verified,
                sb.lastRefresh.Format("15:04:05")))
        }
    } else {
        b.WriteString(fmt.Sprintf(" %d OK | %d %s | %d %s | ",
            sb.activeModels, sb.cooldownCount, sb.sym.CoolDown,
            sb.degradedCount, sb.sym.Degraded))
        if sb.isRefreshing {
            b.WriteString(fmt.Sprintf("%s refreshing",
                sb.sym.Pending))
        } else {
            b.WriteString(fmt.Sprintf("refresh: %s",
                sb.lastRefresh.Format("15:04:05")))
        }
    }

    return b.String()
}

```

8.4 Update Notification Banner (TUI only)

When state changes are detected, a temporary banner appears above the status bar:

 UPDATE: groq-llama-3.1-70b is now available (cooldown cleared)
... main content ...

• 19 models active | " 1 cooldown | 1 degraded | ↓ last refresh

Banner auto-dismisses after 5 seconds or on any keypress.

9. Error / Empty States

9.1 LLMsVerifier Disabled

 HelixCode – Models

⚠ LLMsVerifier is disabled.

Model verification data is unavailable. Showing registered providers on

To enable LLMsVerifier:

1. Set `HELIX_VERIFIER_ENABLED=true` in your environment
2. Or add to `config.yaml`: `llm.verifier.enabled = true`
3. Ensure LLMsVerifier is running at `http://localhost:8081`

[c]ontinue without verification [q]uit

9.2 No Models Pass Validation

 HelixCode – Models

⚠ No verified models available

All registered models failed verification or are pending.

Possible causes:

- API keys are missing or invalid
- Providers are experiencing outages
- Network connectivity issues to provider APIs
- LLMsVerifier verification queue is backed up

3 models pending verification:

-  claude-sonnet-4-5 (Anthropic) – queued 5m ago
-  qwen-2.5-72b (Qwen) – queued 12m ago
-  gemini-2.5-flash (Google) – queued 18m ago

Actions:

[r]etry verification now [v]iew pending details [q]uit

9.3 All Providers in Cooldown



HelixCode – Models

🚫 ALL PROVIDERS IN COOLDOWN

No models are currently available for use.

Cooldown status:

- Groq – rate limited, reset in 12m 34s
- xAI – quota exceeded, reset in 47m 12s
- OpenRouter – temporarily unavailable, reset in 2h 15m

Local providers (no cooldown):

- × Ollama – not running (check <http://localhost:11434>)
- × Llama.cpp – not running (check <http://localhost:8080>)

Suggestions:

1. Wait for rate limits to reset
2. Start a local provider: `ollama serve`
3. Check your API key configurations
4. Use `--provider local` to see only local models

[w]ait and retry in 30s [s]tart local provider [q]uit

9.4 Network to Verifier Down



HelixCode – Models

⚠️ Cannot connect to LLMsVerifier

Connection failed to <http://localhost:8081/api/v1/verifier>
Error: connection refused

Troubleshooting:

1. Is LLMsVerifier running? `./llm-verifier server`
2. Check the verifier URL in `config.yaml`: `llm.verifier.url`
3. Check firewall / port binding

Fallback options:

- [c]ached data (last update: 2026-04-30 14:00) – 23 models
- [o]ffline mode – use only locally registered models
- [r]etry connection [q]uit

9.5 Empty State Implementation

```
// internal/cli/ux/empty_states.go

package ux

func RenderEmptyState(state string, details
    map[string]interface{}, sym *SymbolSet, width int)
    string {
    switch state {
    case "verifier_disabled":
        return renderVerifierDisabled(sym, width)
    case "no_verified_models":
        return renderNoVerifiedModels(details, sym, width)
    case "all_cooldown":
        return renderAllCooldown(details, sym, width)
    case "verifier_unavailable":
        return renderVerifierUnavailable(details, sym, width)
    case "no_models":
        return renderNoModels(sym, width)
    default:
        return renderGenericError(state, sym, width)
    }
}

func renderVerifierDisabled(sym *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CAlertWarning.Sprintf("\n  %s LLMsVerifier is
        disabled.\n\n", sym.Degraded))
    b.WriteString(CValue.Sprint("  Model verification data is
        unavailable. Showing registered providers only.\n\n"))
    b.WriteString(CLabel.Sprint("  To enable LLMsVerifier:\n"))
    b.WriteString(CValue.Sprint("    1. Set
        HELIX_VERIFIER_ENABLED=true in your environment\n"))
    b.WriteString(CValue.Sprint("    2. Or add to config.yaml:
        llm.verifier.enabled = true\n"))
    b.WriteString(CValue.Sprint("    3. Ensure LLMsVerifier is
        running at http://localhost:8081\n\n"))
    b.WriteString(CSubheader.Sprint("  [c]ontinue without
        verification [q]uit\n"))
    return b.String()
}

func renderNoVerifiedModels(details map[string]interface{}, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CAlertWarning.Sprintf("\n  %s No verified
        models available\n\n", sym.Degraded))
}
```



```

b.WriteString(CValue.Sprint("  All registered models failed
    verification or are pending.\n\n"))
b.WriteString(CLabel.Sprint("  Possible causes:\n"))
b.WriteString(CValue.Sprint("    • API keys are missing or
    invalid\n"))

    b.WriteString(CValue.Sprint("    • Providers are
    experiencing outages\n"))
b.WriteString(CValue.Sprint("    • Network connectivity
    issues to provider APIs\n"))
b.WriteString(CValue.Sprint("    • LLMsVerifier verification
    queue is backed up\n\n"))

if pending, ok := details["pending_models"].
    ([]*UnifiedModel); ok && len(pending) > 0 {
    b.WriteString(CLabel.Sprintf("  %d models pending
    verification:\n", len(pending)))
    for _, m := range pending {
        b.WriteString(fmt.Sprintf("    %s %s (%s)\n",
            sym.Pending, m.DisplayName, m.Provider))
    }
    b.WriteString("\n")
}

b.WriteString(CSubheader.Sprint("  [r]etry verification
    now    [v]iew pending details    [q]uit\n"))
return b.String()
}

func renderAllCooldown(details map[string]interface{}, sym
    *SymbolSet, width int) string {
var b strings.Builder
b.WriteString(CAlertError.Sprintf("\n  %s ALL PROVIDERS IN
    COOLDOWN\n\n", sym.QuotaExceeded))
b.WriteString(CValue.Sprint("  No models are currently
    available for use.\n\n"))

if cooldowns, ok := details["cooldowns"].([]*CooldownInfo);
    ok {
    b.WriteString(CLabel.Sprint("  Cooldown status:\n"))
    for _, cd := range cooldowns {
        badge := CooldownBadge(cd.Reason, cd.ResetTime, sym)
        b.WriteString(fmt.Sprintf("    %s %s - %s\n",
            badge.Symbol, cd.ProviderName, badge.Text))
    }
    b.WriteString("\n")
}
}

```

```

b.WriteString(CLabel.Sprint("  Suggestions:\n"))
b.WriteString(CValue.Sprint("    1. Wait for rate limits to
    reset\n"))
b.WriteString(CValue.Sprint("    2. Start a local provider:
    ollama serve\n"))
b.WriteString(CValue.Sprint("    3. Check your API key
    configurations\n"))
b.WriteString(CValue.Sprint("    4. Use --provider local to
    see only local models\n\n"))

    b.WriteString(CSubheader.Sprint("  [w]ait and retry in
    30s    [s]tart local provider    [q]uit\n"))
return b.String()
}

func renderVerifierUnavailable(details map[string]interface{},
    sym *SymbolSet, width int) string {
var b strings.Builder

    b.WriteString(CAlertWarning.Sprintf("\n  %s Cannot
    connect to LLMsVerifier\n\n", sym.Offline))

if url, ok := details["url"].(string); ok {
    b.WriteString(fmt.Sprintf("    Connection failed to %s\n",
        url))
}
if err, ok := details["error"].(string); ok {
    b.WriteString(fmt.Sprintf("    Error: %s\n\n", err))
}

b.WriteString(CLabel.Sprint("  Troubleshooting:\n"))
b.WriteString(CValue.Sprint("    1. Is LLMsVerifier
    running?    ./llm-verifier server\n"))

    b.WriteString(CValue.Sprint("    2. Check the verifier
    URL in config.yaml\n"))
b.WriteString(CValue.Sprint("    3. Check firewall / port
    binding\n\n"))

b.WriteString(CLabel.Sprint("  Fallback options:\n"))
if cached, ok := details["cached_time"].(time.Time); ok {
    b.WriteString(fmt.Sprintf("    [c]ached data (last
        update: %s)\n", cached.Format("2006-01-02 15:04")))
}
b.WriteString(CValue.Sprint("    [o]ffline mode – use only
    locally registered models\n"))

```

```

        b.WriteString(CSubheader.Sprint("      [r]etry connection
        [q]uit\n"))

    return b.String()
}

```

10. Go Structs for UX State Management

10.1 Core UX Package Layout

```

internal/cli/ux/
├── symbols.go           # SymbolSet, TerminalCapabilities, platform det
├── colors.go            # All color definitions, GetScoreColor, GetPric
├── badges.go            # Badge rendering: VerificationBadge, ProviderH
├── capabilities.go      # CapabilityStripCompact, CapabilityStripFull
├── list_display.go      # RenderModelList, table/compact/JSON/CSV rende
├── detail_display.go    # RenderModelDetail, rich/JSON/YAML renderers
├── status_bar.go        # StatusBar component
├── alerts.go            # Alert struct and rendering
├── empty_states.go      # All empty state renderers
├── interactive_fallback.go # Numbered menu for non-TTY
└── model_selector_app.go # TUI app wrapper (delegates to tview)

internal/cli/tui/
└── model_selector.go    # Full tview-based interactive selector

```

10.2 Data Structs

```

// internal/cli/ux/types.go

package ux

import (
    "time"
)

// UnifiedModel — mirrors HelixAgent's UnifiedModel, sourced from
// LLMsVerifier
type UnifiedModel struct {
    ID             string    `json:"id"`
    Name           string    `json:"name"`
    DisplayName    string    `json:"display_name"`
    Provider       string    `json:"provider"`
    Score          float64   `json:"score"`
    Verified       bool      `json:"verified"`
    Latency        time.Duration `json:"latency"`
    ContextWindow  int       `json:"context_window"`
    MaxOutputTokens int       `json:"max_output_tokens"`
    SupportsStreaming bool      `json:"supports_streaming"`
}

```

```

SupportsTools      bool      `json:"supports_tools"`
SupportsFunctions  bool      `json:"supports_functions"`
SupportsVision     bool      `json:"supports_vision"`
SupportsAudio      bool      `json:"supports_audio"`
SupportsVideo      bool      `json:"supports_video"`
SupportsReasoning  bool      `json:"supports_reasoning"`
Capabilities        []string  `json:"capabilities"`
CostPerInputToken  float64  `json:"cost_per_input_token"`
CostPerOutputToken float64  `json:"cost_per_output_token"`
OpenSource         bool      `json:"open_source"`
Deprecated         bool      `json:"deprecated"`
Tags               []string  `json:"tags"`
LanguageSupport    []string  `json:"language_support"`
UseCase            string    `json:"use_case"`
ReleaseDate        string    `json:"release_date"`
Architecture       string    `json:"architecture"`
}

```

```

// UnifiedProvider – mirrors HelixAgent's UnifiedProvider
type UnifiedProvider struct {
    ID            string    `json:"id"`
    Name          string    `json:"name"`
    DisplayName   string    `json:"display_name"`
    Type          string    `json:"type"`
    AuthType      string    `json:"auth_type"`
    Verified      bool      `json:"verified"`
    Score         float64   `json:"score"`
    ScoreSuffix   string    `json:"score_suffix"`
    TestResults   map[string]bool `json:"test_results"`
    CodeVisible   bool      `json:"code_visible"`
    Models        []string  `json:"models"`
    DefaultModel  string    `json:"default_model"`
    Status        string    `json:"status"` // unknown,
    // healthy, degraded, unhealthy, offline
    BaseURL      string    `json:"base_url"`
    Tier         int       `json:"tier"`
    Priority      int       `json:"priority"`
    UptimePct    float64   `json:"uptime_pct"`
    LastHealthCheck time.Time `json:"last_health_check"`
}

```

```

// VerificationResult – mirrors LLMsVerifier VerificationResult
type VerificationResult struct {
    Status        string    `json:"status"` //
    // verified, pending, failed, not_tested
    ModelExists   bool      `json:"model_exists"`
    Responsive    bool      `json:"responsive"`
    Overloaded    bool      `json:"overloaded"`
}

```

```

LatencyMs          int          `json:"latency_ms"`

// Feature flags
SupportsToolUse    bool    `json:"supports_tool_use"`
SupportsCodeGeneration bool    `json:"supports_code_generation"`
SupportsEmbeddings bool    `json:"supports_embeddings"`
SupportsStreaming  bool    `json:"supports_streaming"`
SupportsJSONMode   bool    `json:"supports_json_mode"`
SupportsReasoning  bool    `json:"supports_reasoning"`
SupportsParallelToolUse bool    `json:"supports_parallel_tool_use"`
SupportsBatchProcessing bool    `json:"supports_batch_processing"`
SupportsBrotli     bool    `json:"supports_brotli"`

// Code capabilities
CodeDebugging      bool    `json:"code_debugging"`
CodeOptimization   bool    `json:"code_optimization"`
TestGeneration     bool    `json:"test_generation"`
DocumentationGeneration bool    `json:"documentation_generation"`
Refactoring        bool    `json:"refactoring"`
ErrorResolution     bool    `json:"error_resolution"`
ArchitectureDesign bool    `json:"architecture_design"`
SecurityAssessment  bool    `json:"security_assessment"`
PatternRecognition  bool    `json:"pattern_recognition"`

// Scores (0.0 - 10.0)
OverallScore        float64 `json:"overall_score"`
CodeCapabilityScore float64 `json:"code_capability_score"`
ResponsivenessScore float64 `json:"responsiveness_score"`
ReliabilityScore    float64 `json:"reliability_score"`
FeatureRichnessScore float64 `json:"feature_richness_score"`
ValuePropositionScore float64 `json:"value_proposition_score"`

// Performance
AvgLatencyMs    int    `json:"avg_latency_ms"`
P95LatencyMs    int    `json:"p95_latency_ms"`
MinLatencyMs    int    `json:"min_latency_ms"`
MaxLatencyMs    int    `json:"max_latency_ms"`
ThroughputRps   float64 `json:"throughput_rps"`

Timestamp    time.Time `json:"timestamp"`
Error        string    `json:"error,omitempty"`
}

```

```
// CooldownInfo — tracks rate limit / cooldown state
type CooldownInfo struct {
    ModelID      string      `json:"model_id"`
    ProviderName string      `json:"provider_name"`
    Reason       string      `json:"reason"` // rate-limited,
                quota-exceeded, cooldown, temporarily-unavailable
    ResetTime    time.Time   `json:"reset_time"`
    LimitType    string      `json:"limit_type"`
    LimitValue   int         `json:"limit_value"`
    CurrentUsage int         `json:"current_usage"`
    Message      string      `json:"message,omitempty"`
}
```

```
// RateLimitStatus — current rate limit state for a model
type RateLimitStatus struct {
    ModelID string      `json:"model_id"`
    Limits  []RateLimitEntry `json:"limits"`
}
```

```
type RateLimitEntry struct {
    Type          string      `json:"type"`
    Limit         int         `json:"limit"`
    Used          int         `json:"used"`
    Remaining     int         `json:"remaining"`
    ResetTime     time.Time   `json:"reset_time"`
    IsHardLimit   bool        `json:"is_hard_limit"`
}
```

```
// ModelListRow — a single prepared row for list rendering
type ModelListRow struct {
    Rank          int
    Model         *UnifiedModel
    Provider      *UnifiedProvider
    Verification   *VerificationResult
    Cooldown      *CooldownInfo
}
```

```
// UXState — central state for the model display system
type UXState struct {
    Models        []*UnifiedModel
    Providers     map[string]*UnifiedProvider
    Verifications map[string]*VerificationResult
    Cooldowns     map[string]*CooldownInfo
    RateLimits    map[string]*RateLimitStatus

    LastRefresh    time.Time
    IsRefreshing   bool
    VerifierConnected bool
}
```

```

    VerifierURL    string

    TerminalWidth int
    SymbolSet      *SymbolSet
    NoColor        bool
    NoEmoji        bool
}

func NewUXState() *UXState {
    return &UXState{
        Providers:    make(map[string]*UnifiedProvider),
        Verifications: make(map[string]*VerificationResult),
        Cooldowns:     make(map[string]*CooldownInfo),
        RateLimits:    make(map[string]*RateLimitStatus),
        SymbolSet:
            NewSymbolSet(DetectTerminalCapabilities()),
    }
}

```

10.3 LLMsVerifier Client Struct

```

// internal/verifier/client.go — NEW FILE

package verifier

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "time"
)

const DefaultVerifierURL = "http://localhost:8081/api/v1/
    verifier"

type Client struct {
    baseURL    string
    httpClient *http.Client
}

func NewClient(baseURL string) *Client {
    if baseURL == "" {
        baseURL = DefaultVerifierURL
    }
    return &Client{
        baseURL: baseURL,
        httpClient: &http.Client{Timeout: 30 * time.Second},
    }
}

```

```

}

func (c *Client) HealthCheck(ctx context.Context) error {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/health", nil)
    if err != nil { return err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return err }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("verifier health check failed: %d",
            resp.StatusCode)
    }
    return nil
}

func (c *Client) GetModels(ctx context.Context)
    ([]*ux.UnifiedModel, error) {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/models", nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var models []*ux.UnifiedModel
    if err := json.NewDecoder(resp.Body).Decode(&models); err !=
        nil {
        return nil, err
    }
    return models, nil
}

func (c *Client) GetModel(ctx context.Context, modelID string)
    (*ux.UnifiedModel, error) {
    url := fmt.Sprintf("%s/models/%s", c.baseURL, modelID)
    req, err := http.NewRequestWithContext(ctx, "GET", url, nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var model ux.UnifiedModel
    if err := json.NewDecoder(resp.Body).Decode(&model); err !=
        nil {
        return nil, err
    }
    return &model, nil
}

```



```

}

func (c *Client) GetVerification(ctx context.Context, modelID
    string) (*ux.VerificationResult, error) {
    url := fmt.Sprintf("%s/models/%s/verification", c.baseURL,
        modelID)
    req, err := http.NewRequestWithContext(ctx, "GET", url, nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var result ux.VerificationResult
    if err := json.NewDecoder(resp.Body).Decode(&result); err !=
        nil {
        return nil, err
    }
    return &result, nil
}

func (c *Client) GetProviderStatus(ctx context.Context)
    (map[string]*ux.UnifiedProvider, error) {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/providers", nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var providers map[string]*ux.UnifiedProvider
    if err := json.NewDecoder(resp.Body).Decode(&providers);
        err != nil {
        return nil, err
    }
    return providers, nil
}

func (c *Client) GetRateLimits(ctx context.Context)
    (map[string]*ux.RateLimitStatus, error) {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/limits", nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var limits map[string]*ux.RateLimitStatus

```

```

    if err := json.NewDecoder(resp.Body).Decode(&limits); err !=
        nil {
            return nil, err
        }
    return limits, nil
}

```

11. CLI Flag Additions

11.1 New Flags to Add to cmd/cli/main.go

Replace the existing flag block with the following additions:

```

// Existing flags (KEEP)
command      = flag.String("command", "", "Command to execute")
workerHost   = flag.String("worker", "", "Worker host to add")
workerUser   = flag.String("user", "", "Worker SSH username")
workerKey    = flag.String("key", "", "Worker SSH key path")
model        = flag.String("model", "llama-3-8b",
    "LLM model to use")
prompt       = flag.String("prompt", "", "Prompt for LLM
    generation")
maxTokens    = flag.Int("max-tokens", 1000, "Maximum tokens")
temperature  = flag.Float64("temperature", 0.7, "Generation
    temperature")
stream       = flag.Bool("stream", false, "Stream the
    response")
listWorkers  = flag.Bool("list-workers", false, "List all
    workers")
listModels   = flag.Bool("list-models", false, "List available
    models")
healthCheck  = flag.Bool("health", false, "Perform health
    check")
notify       = flag.String("notify", "", "Send notification")
notifyType   = flag.String("notify-type", "info",
    "Notification type")
notifyPriority = flag.String("notify-priority", "medium",
    "Notification priority")
nonInteractive = flag.Bool("non-interactive", false, "Run in non-
    interactive mode")

// NEW: Model list filter flags
providerFilter = flag.String("provider", "", "Filter models by
    provider name")
verifiedOnly   = flag.Bool("verified-only", false, "Show only
    verified models")

```

```

maxPrice          = flag.Float64("max-price", 0, "Maximum price
    per 1K tokens (0 = no limit)")
minScore          = flag.Float64("min-score", 0, "Minimum overall
    score 0-10 (0 = no limit)")
capabilityFilter   = flag.String("capability", "", "Filter by
    capability: vision,streaming,tools,code,reasoning")
sortBy            = flag.String("sort", "score", "Sort models by:
    score,price,name,provider,latency")
groupBy           = flag.String("group-by", "", "Group models by:
    provider,tier,status")
outputFormat       = flag.String("format", "table",
    "Output format: table,compact,json,csv")
noColor           = flag.Bool("no-color", false, "Disable colored
    output")
noEmoji           = flag.Bool("no-emoji", false, "Disable emoji
    and Unicode symbols")

// NEW: Model detail flag
modelInfo          = flag.String("model-info", "", "Show detailed
    information for a model ID")
modelInfoFormat    = flag.String("model-info-format", "rich",
    "Detail format: rich,json,yaml")

// NEW: Interactive TUI flag
interactiveModels  = flag.Bool("models-interactive", false,
    "Launch interactive model selector TUI")

```

11.2 Flag Wiring in cli.Run()

```

func (c *CLI) Run() {
    flag.Parse()

    ctx := context.Background()

    switch {
    case *listModels:
        c.handleListModels(ctx)
    case *modelInfo != "":
        c.handleModelInfo(ctx, *modelInfo)
    case *interactiveModels:
        c.handleInteractiveModelSelector(ctx)
    case *listWorkers:
        c.handleListWorkers(ctx)
    // ... existing cases ...
    }
}

```

12. Files to Modify / Create

12.1 Files to Create (NEW)

File	Lines	Purpose
internal/cli/ux/symbols.go	~120	SymbolSet, platform detection, fallbacks
internal/cli/ux/colors.go	~60	All color definitions, score/price color helpers
internal/cli/ux/badges.go	~80	Badge rendering functions
internal/cli/ux/capabilities.go	~60	Capability strip rendering
internal/cli/ux/list_display.go	~300	Model list table/compact/JSON/CSV renderers
internal/cli/ux/detail_display.go	~400	Model detail rich/JSON/YAML renderers
internal/cli/ux/status_bar.go	~50	StatusBar component
internal/cli/ux/alerts.go	~60	Alert struct and rendering
internal/cli/ux/empty_states.go	~200	All empty state renderers
internal/cli/ux/interactive_fallback.go	~100	Numbered menu for non-TTY fallback
internal/cli/ux/types.go	~150	Core data structs (UnifiedModel, VerificationResult, etc.)
internal/cli/tui/model_selector.go	~250	Full tview-based interactive selector
internal/verifier/client.go	~150	HTTP client for LLMsVerifier API
internal/config/config_verifier.go	~30	LLMsVerifier config section (add to Config struct)

12.2 Files to Modify (EXISTING)

File	Lines	Change
cmd/cli/main.go	101-128	CRITICAL: Replace hardcoded <code>handleListModel()</code> with dynamic fetch from LLMsVerifier
cmd/cli/main.go	~30-50	Add new CLI flags (see Section 11)
cmd/cli/main.go	~200	

File	Lines	Change
		Add <code>handleModelInfo()</code> , <code>handleInteractiveModelSelector()</code> handlers
<code>cmd/cli/main.go</code>	~250	Wire new flags in <code>cli.Run()</code> switch statement
<code>internal/config/config.go</code>	~260	Add <code>Verifier VerifierConfig</code> field to <code>Config</code> struct
<code>internal/config/config.go</code>	~100	Add verifier defaults in <code>setDefault()</code>
<code>internal/llm/model_discovery.go</code>	~900	Replace <code>fetchExternalModels()</code> hardcoded list with <code>LLMsVerifier</code> fetch
<code>internal/llm/model_manager.go</code>	~280	Add <code>LLMsVerifier</code> status to <code>SelectOptimalModel()</code> scoring
<code>go.mod</code>	-	Add <code>digital.vasic.llmsverifier</code> module dependency

12.3 Config Changes

Add to `internal/config/config.go`:

```

type VerifierConfig struct {
    Enabled    bool    `mapstructure:"enabled"`
    URL        string
        `mapstructure:"url"`           // default: http://
        localhost:8081
    APIKey     string `mapstructure:"api_key"`
    Timeout    int
        `mapstructure:"timeout"`       // seconds, default: 30
    CacheTTL   int
        `mapstructure:"cache_ttl"`     // minutes, default: 5
    AutoRefresh bool
        `mapstructure:"auto_refresh"`  // default: true
}

// In Config struct:
type Config struct {
    // ... existing fields ...
    Verifier VerifierConfig `mapstructure:"verifier"`
}

```

Add defaults in `setDefault()`:

```

viper.SetDefault("verifier.enabled", true)
viper.SetDefault("verifier.url", "http://localhost:8081")
viper.SetDefault("verifier.timeout", 30)

```

```
viper.SetDefault("verifier.cache_ttl", 5)
viper.SetDefault("verifier.auto_refresh", true)
```

13. Implementation Priority

Phase 1: Foundation (Week 1)

1. Create `internal/cli/ux/` package with `symbols.go`, `colors.go`, `types.go`
2. Create `internal/verifier/client.go` for LLMsVerifier API communication
3. Add verifier config to `internal/config/config.go`
4. Create basic `RenderModelList()` for `--list-models` with real data

Phase 2: Core Display (Week 1-2)

1. Implement `handleListModels()` replacement in `cmd/cli/main.go` (P0 — BLUFF-002 fix)
2. Add all new CLI flags to `cmd/cli/main.go`
3. Implement `RenderModelDetail()` for `--model-info`
4. Implement compact/table/JSON/CSV formatters

Phase 3: Advanced Features (Week 2)

1. Implement `CapabilityStrip`, badges, score bars
2. Implement `--provider`, `--verified-only`, `--max-price`, `--min-score`, `--capability` filters
3. Implement `--sort`, `--group-by` options
4. Implement cross-platform symbol fallback system

Phase 4: Interactive Mode (Week 3)

1. Implement `tview`-based `ModelSelectorApp`
2. Implement numbered menu fallback for non-TTY
3. Wire `models` interactive command to TUI

Phase 5: Polish (Week 3-4)

1. Implement real-time status bar
2. Implement alert/notification system
3. Implement all empty states
4. Integration testing with LLMsVerifier
5. Cross-platform testing (Windows `cmd.exe`, PowerShell, macOS, Linux, mobile)

Phase 6: Integration (Week 4)

1. Replace `fetchExternalModels()` hardcoded list in `model_discovery.go`
 2. Add LLMsVerifier status to `SelectOptimalModel()` in `model_manager.go`
 3. Add auto-suggest on unavailable model selection
 4. End-to-end challenge tests
-

Appendix A: Quick Reference — Color to Status Mapping

Status	Color Code	Example
Verified	<code>color.FgHiGreen</code>	✓ 9.4
Pending	<code>color.FgHiYellow</code>	⌚ 7.8
Failed	<code>color.FgHiRed</code>	✗ 4.2
Healthy	<code>color.FgHiGreen</code>	● OpenAI
Degraded	<code>color.FgHiYellow</code>	◐ Groq
Unhealthy	<code>color.FgHiRed</code>	● xAI
Offline	<code>color.FgHiBlack</code>	○ Mistral
Score 9.0+	<code>color.FgHiGreen + bold</code>	9.4
Score 7.0-8.9	<code>color.FgGreen</code>	8.3
Score 5.0-6.9	<code>color.FgYellow</code>	6.5
Score 3.0-4.9	<code>color.FgHiRed</code>	4.2
Score < 3.0	<code>color.FgHiBlack</code>	2.1
Price FREE	<code>color.FgHiCyan + bold</code>	FREE
Price < \$0.50	<code>color.FgHiGreen</code>	\$0.14
Price \$0.50-\$2.00	<code>color.FgYellow</code>	\$1.25
Price > \$2.00	<code>color.FgHiRed</code>	\$15.00

Appendix B: Platform Support Matrix

Platform	Emoji	256 Color	Unicode Box	Recommended SymbolSet
macOS Terminal	✓	✓	✓	Rich
iTerm2	✓	✓	✓	Rich
Linux (GNOME/Konsole)	✓	✓	✓	Rich
Linux (TTY)	✗	✗	✗	ASCII
Windows cmd.exe (Win10)	✗	✗	✗	Windows CMD
Windows cmd.exe (Win11)	✓	✓	✗	ASCII
PowerShell	✓	✓	✓	Rich
Windows Terminal	✓	✓	✓	Rich
WSL	✓	✓	✓	Rich
Aurora OS	✓	✓	✓	Rich
Harmony OS	✓	✓	✓	Rich

Platform	Emoji	256 Color	Unicode Box	Recommended SymbolSet
Android (Termux)	✓	✓	✓	Rich
iOS (iSH/a-Shell)	✓	✓	✓	Rich
CI/Non-TTY	✗	✗	✗	ASCII

Appendix C: API Endpoint Mapping (LLMsVerifier → HelixCode)

LLMsVerifier Endpoint	HelixCode Client Method	Data Used
GET /api/v1/verifier/health	client.HealthCheck()	Connection status
GET /api/v1/verifier/models	client.GetModels()	Model list display
GET /api/v1/verifier/models/{id}	client.GetModel()	Model detail display
GET /api/v1/verifier/models/{id}/verification	client.GetVerification()	Status badges, scores
GET /api/v1/verifier/providers	client.GetProviderStatus()	Provider health, grouping
GET /api/v1/verifier/limits	client.GetRateLimits()	Rate limit panels, cooldown alerts
WS /ws/verifier/events	WebSocket client (future)	Real-time updates

End of UX Design Specification